INTELIGENTNA ANALIZA DANYCH

WYKŁAD NR 2

Kamil Stokfiszewski

Spis treści

1. Sieć MADLINE do rozpoznawania znaków pracująca z plikami graficznymi 2
   1.1 Sposoby reprezentacji i kodowania obrazów 2
   1.2 Wybrane operacje przetwarzania obrazów cyfrowych 8
   1.3 Rysowanie tekstu na obrazach z użyciem czcionek skalowalnych 10
   1.4 Podstawowe operacje obsługi odczytu i zapisu plików tekstowych 12
   1.5 Zadanie projektowe 14
      1.5.1 Przykładowy scenariusz użycia programów projektowych 17
      1.5.2 Specyfikacja zadania projektowego 21
2. Wielowarstwowy perceptron i metoda wstecznej propagacji błędu 22
   2.1 Architektura i sposób działania sieci typu wielowarstwowy perceptron 23
   2.2 Metoda wstecznej propagacji błędu treningu sieci MLP 25

1. Sieć MADLINE do rozpoznawania znaków pracująca z plikami graficznymi

W niniejszym rozdziale zapoznamy się z wersją sieci MADALINE, której budowa omówiona była w materiałach oraz podczas zajęć Zjazdu 1, przystosowaną do pracy z plikami graficznymi. W tym celu najpierw omówione zostaną następujące zagadnienia z zakresu kodowania i przetwarzania obrazów:

- sposoby reprezentacji i kodowania obrazów, w szczególności w aplikacjach języka `Python` z użyciem popularnych bibliotek do przetwarzania obrazów `OpenCV` oraz `PIL`,
- wybrane, podstawowe operacje przetwarzania obrazów, takie jak: (i) konwersja pomiędzy przestrzeniami kolorów obrazu, (ii) progowanie, (iii) binaryzacja, (iv) operacja negatywu, (v) operacja dodania szumu losowego do zawartości obrazu (zaszumienie),
- podstawowe operacje dotyczące manipulacji pojedynczymi pikselami obrazu, umożliwiające realizację takich zadań jak np. rysowanie prostych kształtów,
- rysowanie tekstu na obrazie z użyciem wybranych czcionek skalowanych typu `TrueType`.

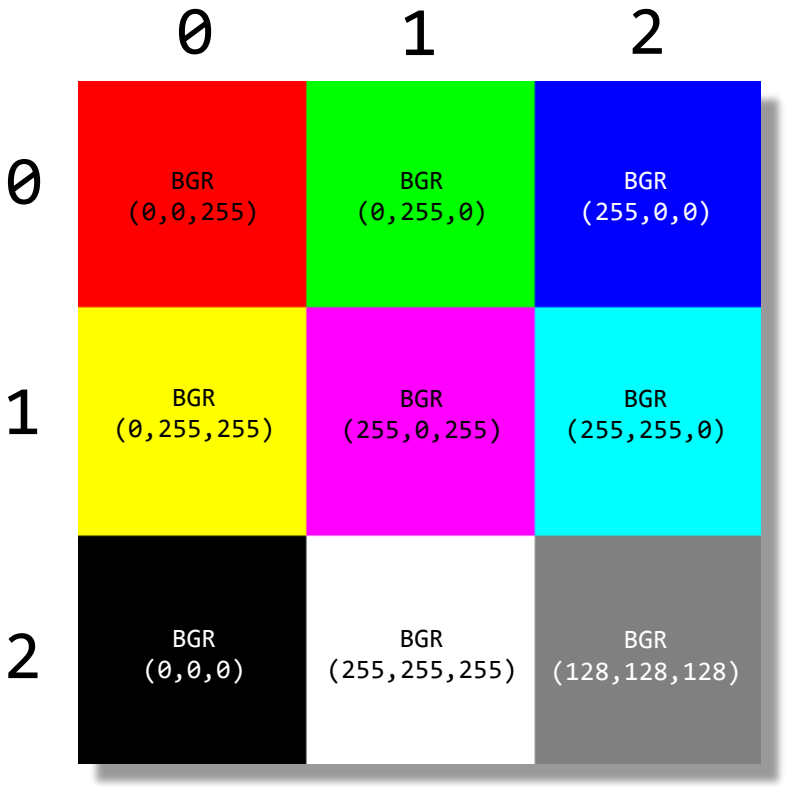
Wymienione wyżej zagadnienia i przykłady zastosowania zaprezentowanych w niniejszej sekcji metod w praktycznych problemach aplikacyjnych, stanowić bazę do realizacji sieci MADALINE przystosowaną do pracy z plikami graficznymi.

1.1 Sposoby reprezentacji i kodowania obrazów

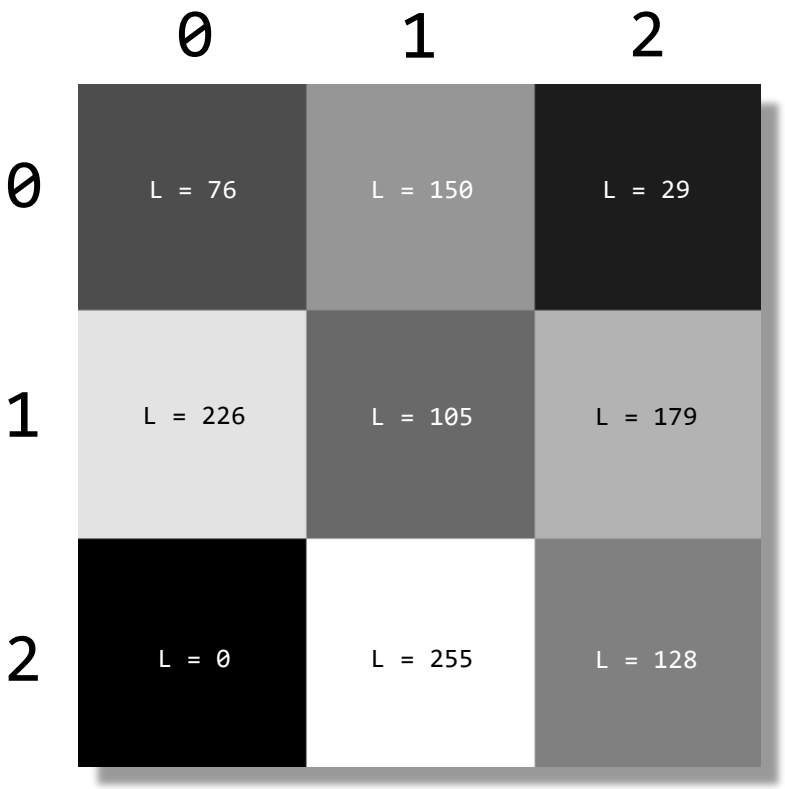
W systemach komputerowych obrazy reprezentowane są najczęściej jako dwuwymiarowe tablice *pikseli*, czyli punktów o zadanym kolorze. W zależności od przyjętego modelu barw istnieją trzy główne rodzaje obrazów:

- obrazy kolorowe,
- obrazy monochromatyczne,
- obrazy binarne.

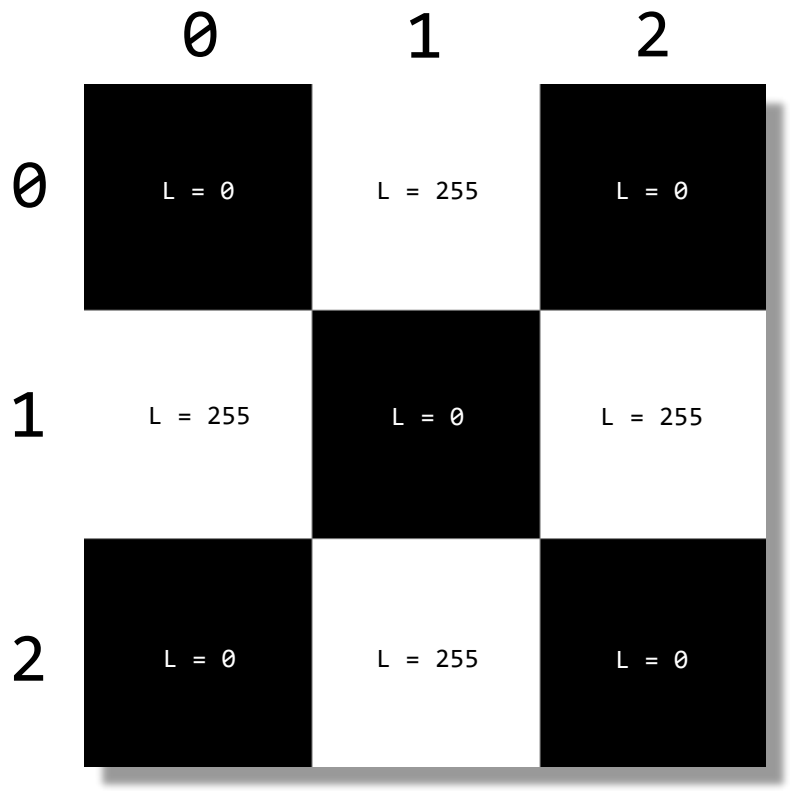
Najbardziej ogólnym rodzajem obrazów są *obrazy kolorowe*. W ich przypadku kolor pojedynczego piksela kodowany jest za pomocą trójki 8-bitowych liczb całkowitych (liczb mogących przyjmować wartości z przedziału od 0 do 255) reprezentujących poszczególne składowe podstawowe barwy danego piksela – składową czerwoną R (ang. *red*), składową zieloną G (ang. *green*) oraz składową niebieską B (ang. *blue*). Wartości pojedynczej składowej koloru decyduje o intensywności danej barwy w kolorze wynikowym danego piksela (wartość minimalna 0 informuje o braku danej barwy w kolorze wynikowym piksela, zaś wartość maksymalna 255 odpowiada za pełną intensywność danej barwy podstawowej w wynikowym kolorze piksela). Z odpowiednich twierdzeń z pogranicza dziedzin fizyki oraz nauki o barwach wiadomo, że wybrana kombinacja trzech barw podstawowych – czerwonej, zielonej i niebieskiej o zadanych intensywnościach pozwala na uzyskanie dowolnego koloru z zakresu widma światła widzialnego (dowolnego koloru z gamy barw tęczy), który może być zarejestrowany przez ludzkie oko. Utworzona w ten sposób przestrzeń kolorów zwana, z racji sposobu kodowania barw, przestrzenią RGB zawiera $2^8 \cdot 2^8 \cdot 2^8 = 2^{24} = 16\,777\,216 \approx 16.7$ miliona, co jest liczbą w zupełności wystarczającą, gdyż badania wskazują, że oko ludzkie jest w stanie rozróżnić około 10 milionów kolorów ze spektrum barw światła widzialnego. Na Rys. 1.1 przedstawiono



Rysunek 1.1. *Sposób reprezentacji obrazu kolorowego.*



Rysunek 1.2. *Sposób reprezentacji obrazu monochromatycznego.*



Rysunek 1.3. *Sposób reprezentacji obrazu binarnego.*

przykład obrazu kolorowego o rozdzielczości 3x3 piksele (odpowiednio w pionie i poziomie) zawierającego różne barwy z przestrzeni kolorów RGB. Z przyczyn czysto technicznych w wielu bibliotekach języków programowania (w tym tych dostępnych dla języka Python) barwy podstawowe definiujące kolor pojedynczego piksela są kodowane jako trójka liczb w kolejności odwrotnej, tzn. jako trójki (*B-blue, G-green, R-red*) – należy o tym pamiętać w trakcie pracy z obrazami kolorowymi. Poniższy Listing 1.1 pokazuje jak w języku Python można utworzyć obraz identyczny jak ten z Rys. 1.1, z użyciem bibliotek programistycznych numpy oraz OpenCV (ang. *open computer vision*) służących do operacji na obrazach, i zapisać go na dysku komputera w postaci pliku graficznego w popularnym formacie png (ang. *portable network graphics*).

```
import cv2
import numpy as np
width  = 3
height = 3
image  = np.zeros((height, width, 3), np.uint8)
image[0][0] = (  0,   0, 255)
image[0][1] = (  0, 255,   0)
image[0][2] = (255,   0,   0)
image[1][0] = (  0, 255, 255)
image[1][1] = (255,   0, 255)
image[1][2] = (255, 255,   0)
image[2][0] = (  0,   0,   0)
image[2][1] = (255, 255, 255)
image[2][2] = (128, 128, 128)
cv2.imwrite('image.png', image)
```

Listing 1.1. *Skrypt w języku Pyhon tworzący obraz kolorowy z Rys. 1.1 i zapisujący go do pliku.*

Jak wynika z Listingu 1.1., obraz tworzony jest w linii 5 jako trójwymiarowa tablica zer biblioteki numpy o pierwszych dwóch wymiarach równych rozdzielczościom, odpowiednio pionowej i poziomej obrazu, i wymiarze trzecim równym 3, odpowiadającym trzem składowym koloru (w kolejności B, G, R) pojedynczego piksela obrazu. Następnie w liniach od 6 do 14 obraz wypełniany jest kolorami pikseli (zgodnie z tymi uwidocznionymi na Rys. 1.1). Wreszcie w linii nr 15, tak utworzony obraz zapisywany jest na dysk komputera za pomocą funkcji `imwrite()` (ang. *image write*) z biblioteki do przetwarzania obrazów `cv2` (OpenCV) języka Python. Warto jeszcze na koniec zwrócić uwagę na ostatni parametr, tzn. `np.uint8` funkcji `np.zeros()` obecny w linii 5 tworzącej obraz – oznacza on, że składowe kolorów poszczególnych pikseli obrazu będą traktowane jako 8-bitowe liczby całkowite bez znaku (ang. *u* – `unsigned`, *int* – `integer`, *8* – `bit`), czyli liczby całkowite z przedziału od `0` do `255`.

Choć obrazy kolorowe, jako te o najbardziej ogólnej reprezentacji, mają szereg zastosowań praktycznych, metody przetwarzania lub/i rozpoznawania obrazów są konstruowane zazwyczaj dla *obrazów monochromatycznych*, gdyż specyficzna informacja o kolorach pikseli w obrazie często nie ma tak istotnego znaczenia w kontekście działania wybranego algorytmu implementującego daną metodę przetwarzania obrazu, jak informacja o jasnościach pikseli. To właśnie ta informacja jest często najistotniejsza z punktu widzenia algorytmów przetwarzania lub/i rozpoznawania obrazów, a ponadto ograniczenie się jedynie do przetwarzania informacji o jasnościach pikseli pozwala algorytmom przetwarzania obrazów na natychmiastową trzykrotną redukcję liczby operacji obliczeniowych wykonywanych na pikselach obrazu, ze względu na fakt, że informacja o jasności danego piksela kodowana jest przy użyciu pojedynczej liczby całkowitej (a nie trójki liczb, tak jak mato miejsce w przypadku obrazów kolorowych). Obrazy monochromatyczne są zatem obrazami o bardzo podobnej strukturze do tych kolorowych, z tą różnicą, że każdemu pikselowi przyporządkowuje się tu pojedynczą 8-bitową liczbę całkowitą (tzn. liczbę z przedziału od `0` do `255`) kodującą jasność tego piksela wg schematu: `0` – jasność minimalna (kolor czarny), `255` – jasność maksymalna (kolor biały) oraz wszystkie liczby pomiędzy tymi wartościami kodujące intensywność odcieni szarości od czarnego do białego (np. piksel o jasności `128`, znajdującej się dokładnie w połowie przedziału `[0,255]` może być interpretowany jako `50%` odcień koloru białego – który w potocznym języku nazywany jest kolorem *szarym*). Na Rys. 1.2 pokazany jest obraz monochromatyczny powstały w wyniku działania następującego skryptu w języku Python:

```
1  import cv2
2  import numpy as np
3  width  = 3
4  height = 3
5  image  = np.zeros((height, width), np.uint8)
6  image[0][0] = 76
7  image[0][1] = 150
8  image[0][2] = 29
9  image[1][0] = 226
10 image[1][1] = 105
11 image[1][2] = 179
12 image[2][0] = 0
13 image[2][1] = 255
14 image[2][2] = 128
15 cv2.imwrite('image.png', image)
```

Listing 1.2. *Skrypt języka Pyhon tworzący obraz monochromatyczny z Rys. 1.2 i zapisujący go do pliku.*

Skrypt z Listingu 1.2 jest niemalże identyczny z tym przedstawionym na Listingu 1.1 dla obrazu kolorowego. Najistotniejsze różnice znajdują się w linii 5, gdzie obraz tworzony jest tym razem jako dwuwymiarowa tablica zer biblioteki numpy o wymiarach równych rozdzielczościom, odpowiednio pionowej i poziomej obrazu. W tym wypadku wartością piksela obrazu jest pojedyncza 8-bitowa liczba całkowita (liczba z przedziału od 0 do 255) kodująca jego jasność. Wartości te ustawiane są w liniach od 6 do 14 skryptu z Listingu 1.2. Warto dodać, że obraz z Rys. 1.2 jest monochromatyczną wersją obrazu z Rys. 1.1, powstałą w wyniku konwersji kolorów z przestrzeni RGB oryginalnego obrazu kolorowego do przestrzeni odcieni szarości (ang. *grayscale*) obrazu wynikowego z użyciem następującego, dobrze znanego wzoru:

$$L_{ij} = 0.299 \cdot R_{ij} + 0.587 \cdot G_{ij} + 0.114 \cdot B_{ij}, \tag{1.1}$$

gdzie: L_{ij} jest wartością odcienia szarości piksela w i-tym wierszu i j-tej kolumnie obrazu wynikowego, zaś wartości R_{ij}, G_{ij} oraz B_{ij} oznaczają poszczególne składowe koloru tego piksela w oryginalnym obrazie źródłowym. Choć konwersji tej można dokonać samodzielnie, to biblioteka OpenCV oferuje gotową funkcję do jej realizacji. Użycie funkcji `cv2.cvtColor()` konwersji obrazu z przestrzeni kolorów RGB do obrazu w skali szarości przedstawia poniższy skrypt:

```
1  import cv2
2  import numpy as np
3  width  = 3
4  height = 3
5  image_color = np.zeros((height, width, 3), np.uint8)
6  image_color[0][0] = (  0,   0, 255)
7  image_color[0][1] = (  0, 255,   0)
8  image_color[0][2] = (255,   0,   0)
9  image_color[1][0] = (  0, 255, 255)
10 image_color[1][1] = (255,   0, 255)
11 image_color[1][2] = (255, 255,   0)
12 image_color[2][0] = (  0,   0,   0)
13 image_color[2][1] = (255, 255, 255)
14 image_color[2][2] = (128, 128, 128)
15 image_gray = cv2.cvtColor(image_color, cv2.COLOR_BGR2GRAY)
16 cv2.imwrite('image_color.png', image_color)
17 cv2.imwrite('image_gray.png',  image_gray)
```

Listing 1.3. *Skrypt języka Pyhon tworzący obraz kolorowy z Rys. 1.1, konwertujący go do obrazu w skali szarości z Rys. 1.2 i zapisujący obydwa obrazy do plików.*

Analizę skryptu z Listingu 1.3, który opiera się na przedstawionych wcześniej skryptach z Listingów 1.1 oraz 1.2 autor pozostawia uczestnikowi kursu.

Ostatnim rodzajem obrazów, które omówimy w niniejszym podpunkcie są *obrazy binarne*, czyli takie, które składają się z pikseli mogących przyjmować tylko dwie wybrane wartości, kodujące dwie możliwe barwy piksela np. czarną i białą. Przykład takiego obrazu podany jest na Rys. 1.3. Choć istnieje możliwość kodowania takich obrazów w postaci bezpośredniej (przy użyciu jednego *bitu* – pojedynczej liczby o wartości 0 lub 1, przeznaczonej do zapamiętania wartości pojedynczego piksela obrazu), to najczęściej w celu kodowania takich obrazów korzysta się, ze względu na jej wygodę, z reprezentacji monochromatycznej, omówionej w poprzednim paragrafie. Wówczas piksel o kolorze czarnym reprezentowany jest wartością 0, zaś

piksel w kolorze białym kodowany będzie z użyciem wartości 255 (zgodnie ze sposobem kodowania odcieni szarości w obrazach monochromatycznych).

Obrazy binarne są szczególnie przydatne we wszelkich metodach rozpoznawania i/lub przetwarzania obrazów, w których najistotniejszą informacją zawartą w obrazie są kształty lub/i kontury elementów graficznych obecnych w obrazie, zaś informacja o ich kolorze czy intensywności odcieni szarości pikseli wchodzących w ich skład jest informacją drugorzędną z punktu widzenia danej metody przetwarzania lub/i rozpoznawania obrazu. W szczególności systemy OCR (ang. *optical character recognition*), służące do rozpoznawania tekstu obecnego w dokumentach skanowanych lub na zdjęciach (np. tablic rejestracyjnych pojazdów czy numerów rejestracyjnych wagonów kolejowych), bardzo często posługują się tego rodzaju obrazami binarnymi. Za bardzo prosty przykład takiego systemu możemy uważać właśnie sieć MADALINE do rozpoznawania znaków, którą analizujemy w ramach niniejszego kursu.

Aby uzyskać obraz binarny z obrazu monochromatycznego można posłużyć się w najprostszym przypadku tzw. *metodą progowania* (uzyskanie obrazu binarnego z obrazu kolorowego może być przeprowadzone poprzez wstępną jego konwersję do obrazu monochromatycznego, zgodnie ze wzorem (1.1), a następnie przez użycie metody progowania w stosunku do tak uzyskanego obrazu monochromatycznego). Progowanie polega na ustawieniu wartości piksela obrazu wynikowego na kolor czarny jeśli odcień szarości odpowiadającego mu piksela w monochromatycznym obrazie źródłowym znajduje się poniżej pewnej, ustalonej dla całego obrazu wartości – tzw. *progu*, np. 128. Biblioteka OpenCV oferuje gotową funkcję o nazwie `cv2.threshold()`, która służy właśnie do sprawnego wykonania operacji progowania w celu uzyskania obrazu binarnego ze źródłowego obrazu monochromatycznego. Przykład użycia tej funkcji obrazuje skrypt języka Python uwidoczniony na Listingu 1.4.

```
 1 import cv2
 2 import numpy as np
 3 width  = 3
 4 height = 3
 5 image_gray = np.zeros((height, width), np.uint8)
 6 image_gray[0][0] = 76
 7 image_gray[0][1] = 150
 8 image_gray[0][2] = 29
 9 image_gray[1][0] = 226
10 image_gray[1][1] = 105
11 image_gray[1][2] = 179
12 image_gray[2][0] = 0
13 image_gray[2][1] = 255
14 image_gray[2][2] = 128
15 _, image_binary  = cv2.threshold(image_gray, 128, 255, cv2.THRESH_BINARY)
16 cv2.imwrite('image_gray.png',   image_gray)
17 cv2.imwrite('image_binary.png', image_binary)
```

Listing 1.4. *Skrypt języka Pyhon tworzący obraz monochromatyczny z Rys. 1.2, konwertujący go do obrazu binarnego z Rys. 1.3 i zapisujący obydwa obrazy do plików.*

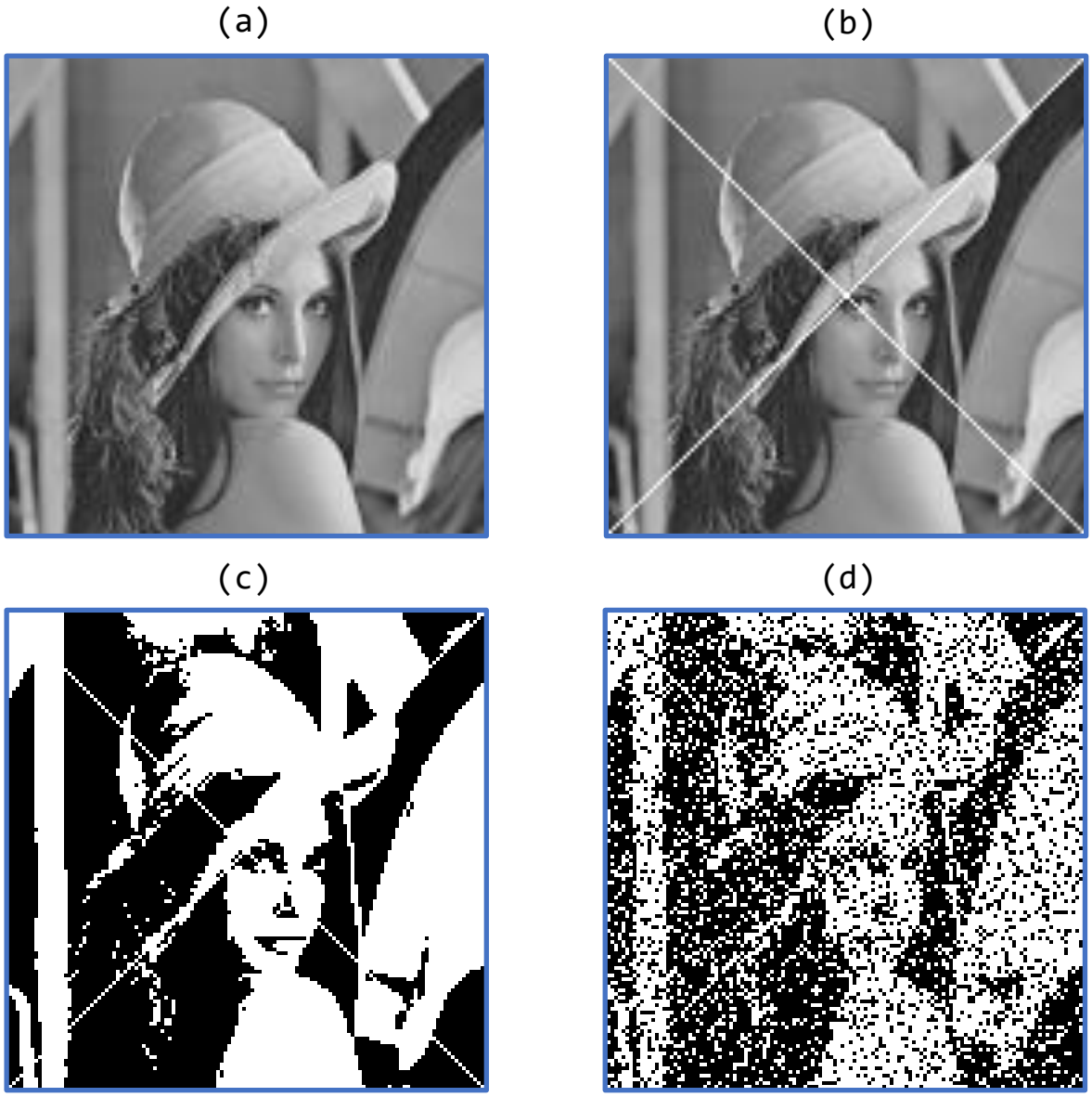
Użycie funkcji progowania uwidocznione jest w linii 15 skryptu z Listingu 1.4. Warto jedynie zwrócić uwagę na fakt, że obraz wynikowy `image_binary` jest zwykłym obrazem monochromatycznym, zawierającym jednak tylko piksele o wartościach `0` i `255`. Ponadto, ze względu na fakt, że funkcja `cv2.threshold()` zwraca dwa obrazy wynikowe (z których pierwszy nie jest istotny dla naszych celów), przypisanie wartości zwracanych przez tę funkcję odbywa się w

sposób następujący: `_, image_binary = cv2.threshold(...)`, gdzie zapis `_,` oznacza, że chcemy zignorować pierwszą wartość zwracaną przez tę funkcję, nie zapamiętując jej w żadnej przeznaczonej do tego celu zmiennej.

Na ostatniej uwadze zakończmy analizę problemu reprezentacji obrazów w systemach komputerowych i przejdźmy do omówienia operacji na obrazach przydatnych z punktu widzenia zadań postawionych przed uczestnikami kursu.

1.2 Wybrane operacje przetwarzania obrazów cyfrowych

W tym podpunkcie przedstawione zostaną wybrane, podstawowe operacje przetwarzania obrazów cyfrowych, przydatne z punktu widzenia dalszych zadań postawionych przed uczestnikami kursu. Spójrzmy na poniższe monochromatyczne obrazy z Rys. 1.4 `(a)-(d)`.



Rysunek 1.4. *Monochromatyczny obraz Lena o rozdzielczości 512x512 pikseli, (a) oryginalny, (b) z narysowanymi liniami, (c) zbinaryzowany, (d) zaszumiony.*

Obrazy te są efektem działania poniższego skryptu języka Python.

```
1  import cv2
2  import copy
3  import numpy as np
4  # image (a)
5  image_gray = cv2.imread('image.png')
6  # image (b)
7  image_draw = copy.deepcopy(image_gray)
8  N = image_draw.shape[0]
9  for i in range(N):
10     image_draw[i][i] = 255
11     image_draw[i][N - i - 1] = 255
12 # image (c)
13 _, image_bin = cv2.threshold(image_draw, 128, 255, cv2.THRESH_BINARY)
14 # image (d)
15 threshold = 20
16 np.random.seed(123)
17 image_noise = copy.deepcopy(image_bin)
18 for i in range(N):
19     for j in range(N):
20         if np.random.rand() <= threshold / 100:
21             image_noise[i][j] = 255 - image_noise[i][j]
22 # save all images to disk
23 cv2.imwrite('image_gray.png',  image_gray)
24 cv2.imwrite('image_draw.png',  image_draw)
25 cv2.imwrite('image_bin.png',   image_bin)
26 cv2.imwrite('image_noise.png', image_noise)
```

Listing 1.5. *Skrypt języka Pyhon tworzący obrazy monochromatyczne z Rys. 1.4 i zapisujący je do plików.*

Prześledźmy kolejne operacje realizowane przez skrypt z Listingu 1.5.

W linii nr 5 funkcją `cv2.imread()` biblioteki OpenCV odczytany zostanie z pliku dyskowego `image.png` wejściowy obraz monochromatyczny. Funkcja `cv2.imread()` sama rozpoznaje rodzaj obrazu (czy jest to obraz kolorowy czy monochromatyczny) i tworzy odpowiednią tablicę modułu numpy (dwu lub trójwymiarową w zależności od rodzaju obrazu odczytanego z pliku – odpowiednio monochromatycznego lub kolorowego). W naszym przykładzie zakładamy, że jest to tablica dwuwymiarowa (obraz monochromatyczny).

W linii 7 robiona jest tzw. *kopia głęboka* tablicy `image_ gray`, która zostaje zapisana w nowej tablicy `image_draw`. Używamy tu funkcji `copy.deepcopy()` modułu copy. Kopia głęboka wykonuje pełną kopię tablicy `image_gray` i zapisuje ją w nowej tablicy. Takie działanie jest potrzebne, gdyż zwykłe przypisanie obiektów tablic reprezentujących obrazy, tzn. wykonanie instrukcji `image_draw = image_gray` języka Python, spowodowałoby utworzenie *referencji* w obiekcie `image_draw` do oryginalnego obiektu tablicowego `image_gray`, co skutkowałoby tym, że obydwie zmienne – zarówno `image_gray`, jak i `image_draw`, reprezentowałyby w rezultacie ten sam obiekt tablicowy modułu numpy. Robienie kopii głębokich obiektów nie będących obiektami wbudowanymi języka Python (takimi jak `int`, `float` czy `str`) jest bardzo ważną czynnością, na którą należy zwracać szczególną uwagę, gdyż w przypadku takich obiektów zwykłe przypisanie zmiennych zawsze skutkuje utworzeniem referencji (czyli tzw. *kopii płytkiej*, ang. *shallow copy*). Na koniec tego wątku należy dodać, że funkcja `copy.deepcopy()` modułu copy języka Python może zostać użyta do tworzenia kopii głębokich obiektów

dowolnego typu i właśnie użycie tej funkcji jest rekomendowanym sposobem realizacji kopii głębokich obiektów języka Python.

W liniach od 8 do 11 skryptu z Listingu 1.5 rysowane są w pętli `for` dwie białe linie przecinające obraz od lewego górnego do prawego dolnego rogu oraz od prawego górnego rogu do lewego dolnego rogu.

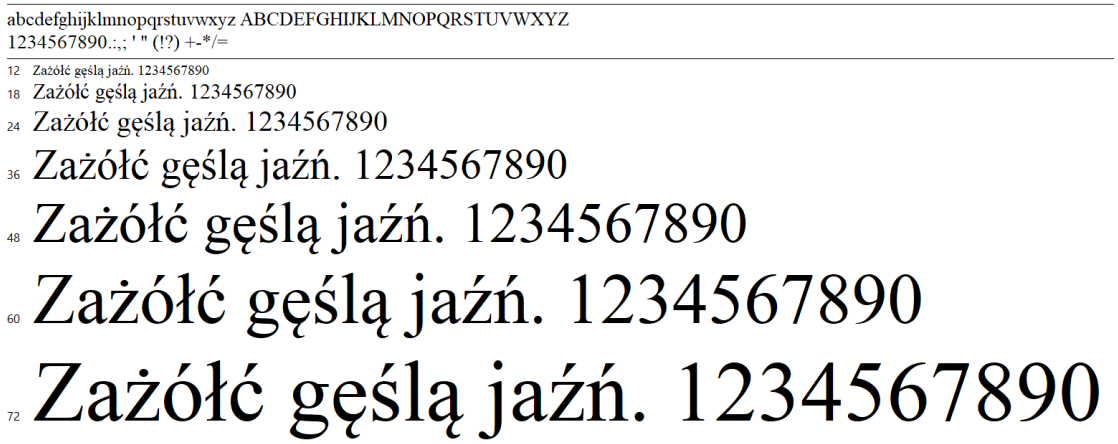
W linii 13 obraz z Rys. 1.4 (`b`) zostaje zbinaryzowany metodą progowania, której działanie zostało szczegółowo przedstawione w poprzednim podpunkcie niniejszego rozdziału.

Linie od 15 do 21 realizują zaszumienie obrazu zbinaryzowanego. Polega ono na zamianie wartości każdego piksela obrazu z czarnej (`255`) na białą (`0`) lub odwrotnie z `20%` prawdopodobieństwem. Prawdopodobieństwo to regulowane jest wartością zmiennej `threshold`, która zdefiniowana jest w linii 15. W celu symulacji procesu losowego zaszumiania obrazu używamy funkcji `np.random.rand()` modułu numpy, która symuluje wybór rzeczywistej liczby losowej z ciągłego przedziału rzeczywistego `[0,1]`. Aby uzyskiwać powtarzalny rezultat symulowanego procesu losowego (tzn. aby za każdym uruchomieniem skryptu obraz zaszumiony wyglądał tak samo) w linii 16 ustawiamy tzw. *ziarno* (ang. *seed*) generatora liczb losowych modułu numpy na pewną wybraną wartość całkowitą (u nas jest to wartość `123`). Usunięcie (lub wykomentowanie) linii 16 analizowanego skryptu spowodowałoby, że za każdym uruchomieniem skryptu otrzymywalibyśmy obraz zaszumiony o innym rozkładzie szumu losowego.

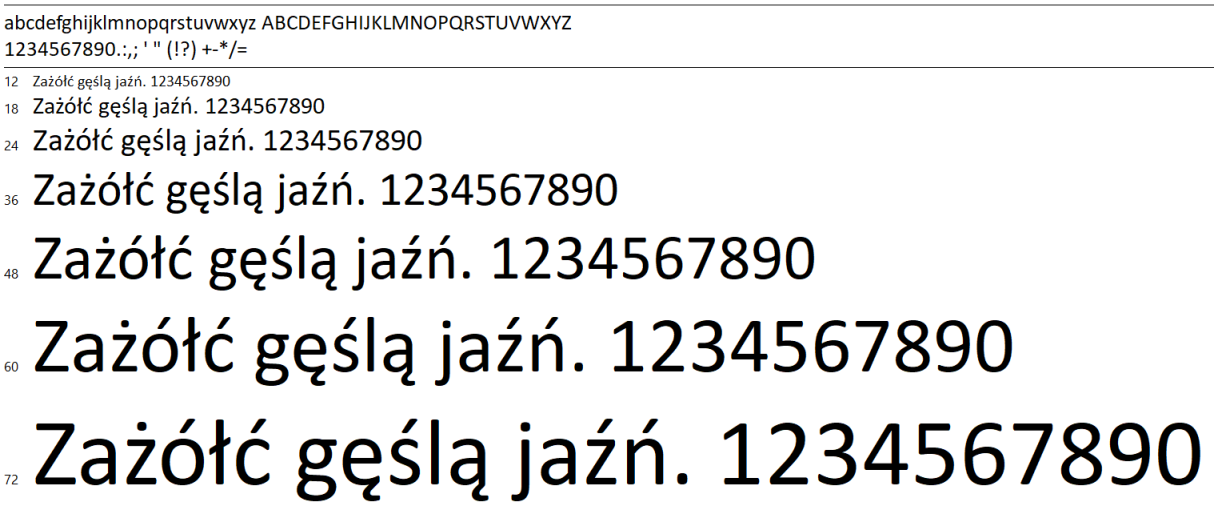
Ostatnie linie skryptu, tzn. linie od 23 do 26 powodują zapis wszystkich czterech przetworzonych obrazów na dysk z odpowiednimi nazwami plików wynikowych.

1.3 Rysowanie tekstu na obrazach z użyciem czcionek skalowalnych

W tym podpunkcie przedstawiona zostanie metoda rysownia tekstu na obrazach z użyciem czcionek skalowalnych typu `TrueType`. Czcionki takie są powszechnie stosowane w cyfrowych dokumentach tekstowych do wektorowego kodowania kształtów liter i znaków alfanumerycznych, tzn. kodowania pozwalającego na dowolne skalowanie wielkości dokumentu bez utraty jego jakości graficznej. Czcionki `TrueType` kodujące kształty wszystkich znaków alfanumerycznych danego zestawu znaków językowych zapisywane są w osobnych plikach z rozszerzeniem `ttf` (od angielskiego określenia *true type font*). Przykłady zestawów czcionek dla kroju pism `Times New Roman` oraz Calibri przedstawione są na Rys. 1.5 oraz 1.6.



Rysunek 1.5. *Próbki kształtów liter oraz znaków alfanumerycznych dla różnych rozmiarów znaków skalowalnej czcionki typu `TrueType` w kroju pisma `Times New Roman`.*

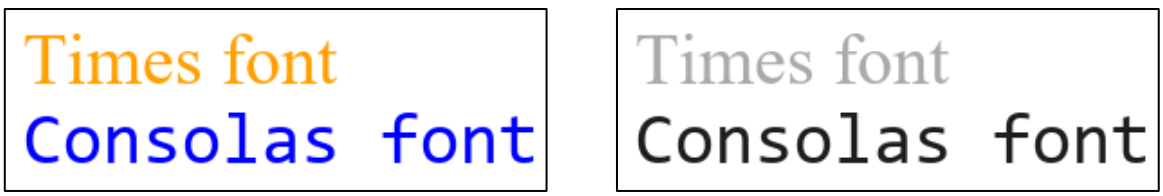


Rysunek 1.6. *Próbki kształtów liter oraz znaków alfanumerycznych dla różnych rozmiarów znaków skalowalnej czcionki typu TrueType w kroju pisma Calibri.*

Biblioteki przetwarzania obrazów dostępne dla języka Python pozwalają na stosunkowo łatwe generowanie obrazów zawierających tekst z użyciem czcionek typu True Type. Spójrzmy na kod źródłowy poniższego skryptu.

```
import cv2
import numpy as np
from PIL import Image, ImageFont, ImageDraw
width  = 300
height = 100
image  = Image.new('RGB', (width, height), (255, 255, 255))
draw   = ImageDraw.Draw(image)
x = 10; y = 5;
font = ImageFont.truetype('times.ttf', 40)
draw.text((x, y), 'Times font', font = font, fill = (0, 160, 255))
x = 10; y = 55;
font = ImageFont.truetype('consolas.ttf', 40)
draw.text((x, y), 'Consolas font', font = font, fill = (255, 0, 0))
cv_color_image = np.asarray(image)
cv2.imwrite('color_image.png', cv_color_image)
cv_gray_image = cv2.cvtColor(cv_color_image, cv2.COLOR_BGR2GRAY)
cv2.imwrite('gray_image.png', cv_gray_image)
```

Listing 1.6. *Skrypt języka Pyhon tworzący obrazy monochromatyczne z Rys. 1.4 i zapisujący je do plików.*



Rysunek 1.7. *Obrazy kolorowy i monochromatyczny uzyskane w wyniku działania skryptu z Listingu 1.6.*

Na Rys. 1.7 znajdują się obrazy kolorowy (color_image.png) i monochromatyczny (gray_image.png) uzyskane w wyniku działania skryptu z Listingu 1.6. Skrypt ten wykorzystuje bibliotekę PIL (*Python Image Library*) języka Python służącą do realizacji prostych operacji przetwarzania obrazów. Choć biblioteka ta zdecydowanie mniejsze możliwości niż rozbudowana OpenCV, to jednak w mniej skomplikowanych zadaniach, takich jak np. rysowanie tekstu, kod źródłowy napisany z jej wykorzystaniem wydaje się znacznie prostszy.

Program z Listingu 1.6 najpierw, przy użyciu funkcji biblioteki PIL, tworzy obraz kolorowy o rozdzielczości 300x100 pikseli, wypełniony białym tłem (linia 6). Następnie tworzony jest obiekt Draw skojarzony z tym obrazem, służący do realizacji operacji rysowania. W linii 9 tworzony jest obiekt typu Font, reprezentujący czcionkę TrueType o rozmiarze 40 punktów, która będzie używana do rysowania tekstu. Czcionka ta wczytywana jest z pliku kształtów czcionek times.ttf, który znajduje się w katalogu roboczym z którego uruchamiany jest skrypt (pliki krojów czcionek można znaleźć np. w komputerach z zainstalowanym systemem Windows 10 lub starszymi, w katalogu systemowym C:\Windows\Fonts). Następnie (linia 10) funkcja text(), wywołana na rzecz obiektu draw, dokonuje wyrysowania tekstu 'Times font' na tle wcześniej utworzonego obrazu w pozycji (10,5) punktów we współrzędnych obrazu, z użyciem wczytanej czcionki, wypełnionego kolorem (0,160,255) zapisanym we współrzędnych BGR (kolorem *ecru*). Linie od 11 do 13 analizowanego skryptu wykonują analogiczne operacje rysowania tekstu dla czcionki o kroju Consolas (tutaj, podobnie jak wcześniej, plik kroju czcionek consolas.ttf powinien znajdować się w katalogu roboczym z którego uruchamiany jest skrypt).

Następnie, w linii 14, obiekt image obrazu biblioteki PIL jest konwertowany do dobrze już znanej reprezentacji tablicowej obrazu kolorowego biblioteki numpy za pomocą funkcji np.asarray() tej biblioteki. Od tego momentu na obrazie można dokonywać już dowolnych operacji w sposób poznany w poprzednim punkcie podczas omawiania operacji na obrazach z użyciem biblioteki OpenCV. W naszym przypadku uzyskany obraz kolorowy zapisywany jest natychmiast do pliku o nazwie color_image.png, a następnie konwertowany do obrazu monochromatycznego za pomocą funkcji cv2.cvtColor() biblioteki OpenCV i zapisywany do pliku o nazwie gray_image.png.

W ten sposób poznaliśmy już prawie wszystkie aspekty potrzebne do realizacji prostego systemu rozpoznawania znaków opartego o wykorzystanie sieci MADALINE i wykorzystującego pliki graficzne. Pozostaje jeszcze tylko omówić krótko podstawowe operacje obsługi odczytu i zapisu plików tekstowych w języku Python, gdyż pliki te będą wykorzystywane w charakterze plików konfiguracyjnych dla budowanego przez nas systemu.

1.4 Podstawowe operacje obsługi odczytu i zapisu plików tekstowych

W niniejszym podpunkcie przedstawione zostaną podstawowe operacje obsługi odczytu i zapisu plików tekstowych w języku Python, przydatne podczas budowy prostego systemu rozpoznawania znaków opartego o wykorzystanie sieci neuronowej typu MADALINE. Spójrzmy najpierw na poniższy skrypt w języku Python demonstrujący realizację podstawowych operacji zapisu i odczytu danych do i z plików tekstowych.

Pierwsza część skryptu demonstruje wykonanie operacji zapisu do pliku tekstowego. W linii 2 plik o nazwie file.txt jest najpierw otwierany do zapisu, co określone jest drugim parametrem funkcji open() otwarcia pliku – parametr 'w' oznacza otwarcie pliku tekstowego w trybie do zapisu (ang. *write*), jeśli plik ten nie istnieje zostanie utworzony. Następnie na rzecz zmiennej plikowej file wykonywane są zapisy kolejnych linii pliku, przy czym koniec aktualnej

linii pliku kodowany jest znakiem końca linii `'\n'`(ang. *newline character*). Po zakończeniu zapisu wszystkich linii do otwartego pliku tekstowego należy go zamknąć wywołując funkcję `close()` na rzecz zmiennej plikowej `flie` reprezentującej przetwarzany plik (linia 9).

```
 1 # write operations part
 2 file = open('file.txt', 'w')
 3 file.write('1st line\'s label')
 4 file.write(':')
 5 file.write('1st line\'s description\n')
 6 file.write('2nd line\'s label:')
 7 file.write('2nd line\'s description\n')
 8 file.write('3rd line\'s label:3rd line\'s description\n')
 9 file.close()
10 # read operations part
11 file  = open('file.txt', 'r')
12 lines = file.read().splitlines()
13 file.close()
14 for i in range(len(lines)):
15     line  = lines[i]
16     words = line.split(':')
17     label = words[0]
18     description = words[1]
19     print('line = %d, label = \"%s\", description = \"%s\"' % \
20           (i, label, description))
```

Listing 1.7. *Skrypt języka Pyhon demonstrujący operacje zapisu i odczytu plików tekstowych.*

Wynikiem działania pierwszej części skryptu jest utworzenie na dysku komputera pliku tekstowego o nazwie `file.txt` i następującej zawartości:

```
1 1st line's label:1st line's description
2 2nd line's label:2nd line's description
3 3rd line's label:3rd line's description
```

Listing 1.8. *Zawartość pliku tekstowego utworzonego w wyniku działania pierwszej części skryptu 1.7.*

Druga część skryptu demonstruje wykonanie operacji odczytu danych z utworzonego w ten sposób pliku tekstowego. W linii 11 plik jest najpierw otwierany do odczytu, co określone jest drugim parametrem funkcji `open()` otwarcia pliku – parametr `'r'` oznacza otwarcie pliku tekstowego w trybie do odczytu (ang. *read*). Następnie całość pliku jest odczytywana do listy łańcuchów tekstowych `lines` za pomocą sekwencji wywołań funkcji `read().splitlines()` realizowanych na rzecz zmiennej plikowej `file`. Lista `lines` jest teraz jednowymiarową listą łańcuchów, z których każdy reprezentuje pojedynczą linię zapisaną w pliku `file.txt`, przy czym łańcuchy te nie zawierają znaków końca linii `'\n'`. Po odczytaniu zawartości całego pliku i utworzeniu listy `lines`, plik jest zamykany w linii 13 funkcją `close()`.

Ostatnia część skryptu, zawarta w liniach od 14 do 20, demonstruje przykładowy sposób interpretacji tekstu zawartego w kolejnych liniach listy `lines`. Zakładamy tu prostą składnię własnego pomysłu każdej linii odczytanego pliku składającą się z etykiety linii (ang. *label*) oraz opisu (ang. *description*) związanego z tą etykietą. Obydwie te części pojedynczej linii oddzielone są znakiem dwukropka `':'` pełniącego rolę tzw. *separatora*. W pętli `for`, zawartej w liniach od 14 do 20, przechodzimy po wszystkich liniach odczytanych z pliku do listy `lines`

i dla każdej z tych linii rozdzielamy ją na części względem znaku separatora korzystając z funkcji `split()` wywołanej na rzecz łańcucha `line` reprezentującego pojedynczą linię zawartą w liście `lines`. Obydwie części pojedynczej linii, tzn. łańcuch reprezentujący etykietę i łańcuch reprezentujący opis, trafiają najpierw do listy łańcuchów `words` (linia 16), aby ostatecznie znaleźć się w odpowiadających im zmiennych łańcuchowych `label` oraz `description` (linie 17 oraz 18). Następnie, w liniach 19 i 20, instrukcja `print()` wyświetla odpowiednio sformatowany tekst opisujący dane, które udało się uzyskać na podstawie analizy tekstu zawartego w pojedynczej linii odczytanego pliku – tzn. nr linii, etykietę zawartą w linii oraz jej opis. Listing 1.9 przedstawia wynik działania skryptu 1.7 w postaci komunikatu wyświetlonego w konsoli języka Python po jego wykonaniu.

```
1 line = 0, label = "1st line's label", description = "1st line's description"
2 line = 1, label = "2nd line's label", description = "2nd line's description"
3 line = 2, label = "3rd line's label", description = "3rd line's description"
```

Listing 1.9. *Wynik działania skryptu 1.7 w postaci komunikatu wyświetlonego w konsoli języka Python.*

Teraz znamy już wszystkie aspekty potrzebne do realizacji prostego systemu rozpoznawania znaków opartego o wykorzystanie sieci MADALINE, wykorzystującego pliki graficzne oraz tekstowe pliki konfiguracyjne. Podajmy zatem treść zadania projektowego.

1.5 Zadanie projektowe

W niniejszym podpunkcie przedstawiona zostanie treść zadania projektowego zmierzającego do realizacji prostego systemu rozpoznawania znaków opartego o wykorzystanie sieci neuronowej typu MADALINE i wykorzystującego w tym celu pliki graficzne oraz tekstowe pliki konfiguracyjne. Zadanie składa się z realizacji dwóch programów (skryptów) narzędziowych:

- programu `font_generator.py`, służącego do generowania plików graficznych zawierających próbki czcionek przeznaczonych do późniejszego rozpoznawania, reprezentujących pojedyncze litery o rożnym stopniu zaszumienia obrazu,

- programu `madaline_ocr.py`, służącego do rozpoznawania pojedynczych próbek czcionek na podstawie obrazów wygenerowanych za pomocą skryptu `font_generator.py`.

Przejdźmy najpierw do omówienia funkcjonalności programu `font_generator.py`. Program ten powinien być skryptem języka Python, uruchamianym z linii poleceń systemu operacyjnego (lub konsoli czy notatnika języka Python w zależności od użytego środowiska programistycznego) z podanymi niżej parametrami o następującej składni:

```
font_generator.py w h x y font_file letter noise_level output_directory
```

gdzie:

- `w` oraz `h` oznaczają, odpowiednio szerokość i wysokość generowanego obrazu,
- `x` oraz `y` oznaczają pozycję pojedynczej litery na wygenerowanym obrazie,
- `font_file` jest nazwą pliku `ttf` zawierającego krój czcionki obrazu litery,

- noise_level jest liczbą całkowitą określającą w procentach poziom zaszumienia wygenerowanego obrazu litery przeznaczonej do rozpoznania,
- output_directory jest ścieżką katalogu docelowego, w którym powinien zostać zapisany plik graficzny z wygenerowanym obrazem litery oraz plik konfiguracyjny zawierający informacje na temat wygenerowanych obrazów liter przeznaczone dla programu rozpoznającego.

Program font_generator.py służy w zamyśle do zautomatyzowania i ułatwienia procesu generowania obrazów liter, przeznaczonych do późniejszego rozpoznania, z użyciem wybranych czcionek TrueType, pozwalającym na generowanie obrazów binarnych o różnym poziomie zaszumienia. Umożliwia to w perspektywie przetestowanie skuteczności działania algorytmu rozpoznawania znaków z użyciem sieci MADALINE dla obrazów liter o różnym poziomie zniekształceń. Wszystkie aspekty techniczne związane z realizacją programu zostały omówione szczegółowo w podrozdziałach 1.1 do 1.4 niniejszego rozdziału.

Podczas pojedynczego uruchomienia program powinien generować plik zawierający obraz binarny o rozdzielczości w na h pikseli, który w pozycji (x,y) zawiera kształt pojedynczej litery podanej parametrem letter o kroju czcionki TrueType odczytanym z pliku ttf o nazwie podanej parametrem font_file. Kształt litery powinien być obrazem zaszumionym w (podanym procentowo) stopniu, określonym wartością parametru noise_level (który w przypadku wartości 0 oznacza obraz niezaszumiony). Wygenerowany w ten sposób obraz powinien być zapisany do pliku graficznego w formacie png, pod nazwą letter.png (np. a.png, czy b.png lub x.png) znajdującego się w folderze, którego ścieżka określona jest parametrem output_directory. Ponadto, oprócz pliku graficznego, program powinien do tego samego folderu zapisywać plik tekstowy o nazwie description.txt (lub dopisywać do tego pliku jeśli już istnieje w podanym folderze[1]) pojedynczą linię konfiguracyjną o następującym formacie:

```
[letter].png:letter [letter], noise level [noise_level]%'\n'
```

gdzie identyfikatory podane w nawiasach oznaczają parametry z którymi wywołany został program, zaś '\n' jest znakiem nowej linii. Przykładowa linia konfiguracyjna mogłaby mieć następującą postać:

```
a.png:letter a, noise level 20%
```

lub np.:

```
x.png:letter x, noise level 0%
```

Pierwsza część linii konfiguracyjnej zawiera nazwę pliku graficznego z wygenerowanym (oraz ewentualnie zaszumionym) obrazem wybranej litery, zaś po znaku separatora (którym tutaj jest dwukropek) znajduje się tekstowy opis zawartości obrazu z kształtem litery, który będzie wykorzystywany później przez program madaline_ocr.py do prezentacji wyników rozpoznawania.

[1] Dopisywanie do istniejącego już pliku tekstowego kolejnych linii jest realizowane identycznie jak zapis do pliku (omówiony w punkcie 1.4) z tą różnicą, że należy otworzyć plik o wybranej nazwie w trybie do dopisywania, z drugim parametrem funkcji open() równym wartości 'a' (ang. *append*) zamiast 'w' (tak, jak to ma miejsce w przypadku otwierania plików do zapisu). Ponadto otwarcie pliku w trybie do dopisywania spowoduje utworzenie tego pliku na dysku jeśli wcześniej plik ten nie istniał.

Przejdźmy teraz do opisu funkcjonalności programu do rozpoznawania znaków `madaline_ocr.py`. Podobnie jak w przypadku poprzedniego skryptu, program `madaline_ocr.py` powinien być skryptem języka Python, uruchamianym z linii poleceń systemu operacyjnego z podanymi niżej parametrami wywołania o następującej składni:

```
madaline_ocr.py train_directory test_directory
```

gdzie parametr `train_directory` jest ścieżką do katalogu, w którym znajdują się pliki graficzne png, zawierające kształty liter wzorcowych służących do konstrukcji i wyznaczenia wag sieci MADLINE do rozpoznawania znaków. W katalogu tym powinien znajdować się także pojedynczy tekstowy plik konfiguracyjny `description.txt`, wygenerowany wcześniej przez program `font_generator.py`, w którym znajduje się lista plików png, znajdujących się w katalogu `train_directory`, wraz z opisami zawartości każdego z tych plików (tzn. etykietami odpowiadającymi każdemu z plików z kształtem litery), które posłużą do prezentacji wyników rozpoznawania. Parametr `test_directory`, ma bardzo podobne znaczenie, co opisany przed chwilą parametr `train_directory`. Katalog `test_directory` powinien mieć taką samą strukturę, co folder `train_directory`, z tą różnicą, że w katalogu tym powinny znajdować się pliki graficzne png, zawierające kształty liter testowych, które będą podlegać rozpoznawaniu. Powinien on zawierać także pojedynczy tekstowy plik konfiguracyjny `description.txt`, o identycznym znaczeniu, co to, opisane wyżej dla przypadku folderu `train_directory`.

Po uruchomieniu, program `madaline_ocr.py` powinien odczytać pliki graficzne z kształtami liter wzorcowych, zawarte w katalogu `train_directory` (korzystając z pliku `description.txt`) i na ich podstawie zbudować sieć MADALNE, w taki sposób, jaki został opisany w materiałach do zjazdu nr 1. Każdemu neuronowi odpowiadającemu za rozpoznanie pojedynczego kształtu litery powinna być ponadto przypisana etykieta, odczytana również z pliku konfiguracyjnego `description.txt`, znajdującego się w katalogu `train_directory`.

Po zbudowaniu sieci program `madaline_ocr.py` powinien przystąpić do rozpoznawania znaków testowych zapisanych w plikach graficznych png, zawartych w katalogu `test_directory`. Podobnie jak wcześniej lista nazw plików testowych oraz ich etykiety powinny zostać odczytane z pliku konfiguracyjnego `description.txt`, znajdującego się w katalogu `test_directory`. Dla każdego ze znaków testowych powinien zostać uformowany wektor wejściowy do sieci MADALINE, który następnie powinien zostać poddany propagacji do jej warstwy wyjściowej, odpowiadającej za właściwe rozpoznanie znaku. Następnie program powinien zidentyfikować indeks neuronu zwycięskiego odpowiadającego rozpoznanej literze testowej. Po dokonaniu wspomnianej identyfikacji program powinien wyświetlić w konsoli komunikat dotyczący wyniku rozpoznawania o następującym formacie:

```
[test_pattern_label] -> [train_pattern_label], confidence: [cnf]'\n'
```

gdzie `test_pattern_label` jest tekstem etykiety przyporządkowanym aktualnie rozpoznawanemu znakowi testowemu (tekst ten pochodzi z pliku description.txt opisującego znaki testowe), `train_pattern_label` jest tekstem etykiety przyporządkowanym neuronowi zwycięskiemu, odpowiadającemu rozpoznanemu znakowi (tekst ten pochodzi z pliku description.txt opisującego znaki wzorcowe), zaś liczba rzeczywista `cnf`, jest maksymalną wartością uzyskaną na wyjściach sieci dla neuronu zwycięskiego, odpowiadającą *współczynnikowi pewności* rozpoznania znaku (tak jak poprzednio `'\n'` jest znakiem nowej linii). Nie należy przy tym mylić wartości współczynnika pewności z pewnością rozpoznania znaku wyrażoną np. w procentach. Choć współczynnik ten będzie miał zawsze wartość mieszczącą się w przedziale rzeczywistym

[0,1], to jednak, pamiętając rozważania teoretyczne ze zjazdu nr 1, jego wartością jest wartość funkcji *cosinus* kąta zawartego pomiędzy znormalizowanym wektorem wagowym neuronu zwycięskiego, reprezentującym rozpoznany znak, a znormalizowanym wektorem wejściowym sieci, reprezentującym znak testowany – tak i tylko tak wartość ta powinna być interpretowana. Czynności opisane przed chwilą w poprzednim paragrafie powinny zostać powtórzone dla każdego z obrazów kształtów znaków zawartych w katalogu testowym `test_directory`. Przykładowe wyjście konsoli programu `madaline_ocr.py` po jego wykonaniu mogłoby wyglądać w następujący sposób:

```
letter a, noise level 10% -> letter a, noise level 0%, confidence: 0.829
letter b, noise level 10% -> letter b, noise level 0%, confidence: 0.872
letter c, noise level 10% -> letter c, noise level 0%, confidence: 0.807
letter a, noise level 70% -> letter a, noise level 0%, confidence: 0.382
letter b, noise level 70% -> letter b, noise level 0%, confidence: 0.420
letter c, noise level 70% -> letter c, noise level 0%, confidence: 0.342
letter a, noise level  0% -> letter a, noise level 0%, confidence: 1.000
letter b, noise level  0% -> letter b, noise level 0%, confidence: 1.000
letter c, noise level  0% -> letter c, noise level 0%, confidence: 1.000
```

Aby rozwiać wszelkie wątpliwości dotyczące funkcjonalności obydwu programów projektowych, podajmy przykładowy scenariusz ich użycia.

1.5.1 Przykładowy scenariusz użycia programów projektowych

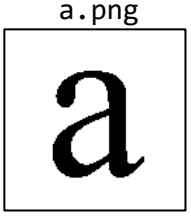
Załóżmy, że chcemy skonstruować sieć MADALINE zdolną do rozpoznawania trzech znaków odpowiadających literom alfabetu angielskiego 'a', 'b' oraz 'c' o kształtach odpowiadających krojom czcionki *Times New Roman*, zawartym w pliku o nazwie `times.ttf`.

W tym celu tworzymy najpierw odpowiednie podkatalogi: `training_patterns` oraz `test_patterns_30` oraz `test_patterns_70`, w folderze, w którym znajdują się obydwa napisane przez nas skrypty projektowe `font_generator.py` oraz `madaline_ocr.py`. Do katalogu tego kopiujemy również plik opisu kształtu czcionki `times.ttf`. Uruchamiamy następnie trzykrotnie program `font_generator.py`, aby wygenerować pliki obrazów znaków wzorcowych w następujący sposób:

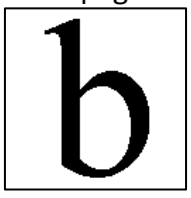
```
font_generator.py 128 128 28 10 times.ttf a 0 training_patterns
font_generator.py 128 128 28 10 times.ttf b 0 training_patterns
font_generator.py 128 128 28 10 times.ttf c 0 training_patterns
```

Po trzykrotnym wykonaniu programu `font_generator.py` z podanymi wyżej parametrami linii poleceń, w podkatalogu `training_patterns` powinny znaleźć się trzy pliki wyjściowe o rozdzielczościach 128x128 pikseli każdy, reprezentujące obrazy liter wzorcowych, które mogłyby wyglądać np. w następujący sposób:

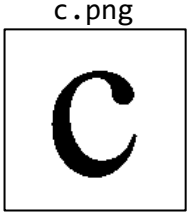
a.png



b.png



c.png



(zauważmy, że parametry wywołania noise_level każdego z trzech wykonań programu font_generator.py są równe 0, co oznacza, że generujemy znaki bez dodawania do nich szumu losowego, czyli znaki „czyste"). Ponadto, w tym samym podkatalogu training_patterns, w wyniku wszystkich trzech wywołań programu font_generator.py wygenerowany zostanie pojedynczy tekstowy plik konfiguracyjny description.txt, który, zgodnie z opisaną wyżej specyfikacją, będzie miał następującą postać:

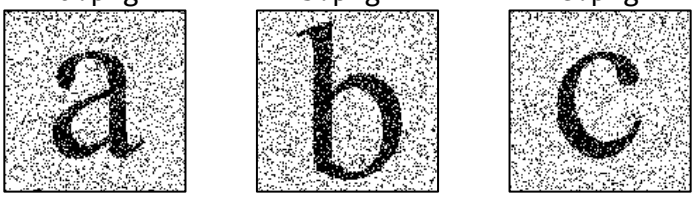
Zawartość pliku tekstowego description.txt w podkatalogu training_patterns:
```
a.png:letter a, noise level 0%
b.png:letter b, noise level 0%
c.png:letter c, noise level 0%
```

Następnie wykonujemy analogiczne czynności, aby wygenerować pliki testowe. Tutaj uruchamiamy również trzykrotnie program font_generator.py, aby wygenerować pliki testowe, przeznaczone do przetestowania skuteczności procesu rozpoznawania znaków o 30% poziomie zaszumienia:

```
font_generator.py 128 128 28 10 times.ttf a 30 test_patterns_30
font_generator.py 128 128 28 10 times.ttf b 30 test_patterns_30
font_generator.py 128 128 28 10 times.ttf c 30 test_patterns_30
```

Po trzykrotnym wykonaniu programu font_generator.py z podanymi wyżej parametrami linii poleceń, w podkatalogu test_patterns_30 powinny znaleźć się trzy pliki wyjściowe o rozdzielczościach 128x128 pikseli każdy, reprezentujące obrazy liter testowych o 10% poziomie zaszumienia losowego, które mogłyby wyglądać np. w następujący sposób:

a.png b.png c.png



Ponadto, w tym samym podkatalogu test_patterns_30, w wyniku wszystkich trzech wywołań programu font_generator.py wygenerowany zostanie pojedynczy tekstowy plik konfiguracyjny description.txt, który, będzie miał następującą postać:

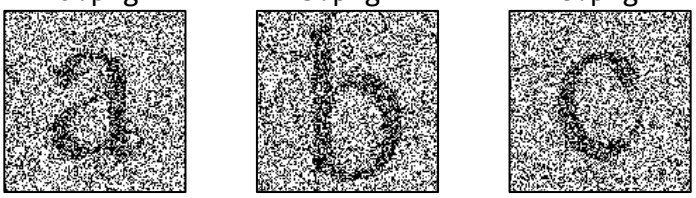
Zawartość pliku tekstowego description.txt w podkatalogu test_patterns_30:
```
a.png:letter a, noise level 30%
b.png:letter b, noise level 30%
c.png:letter c, noise level 30%
```

Wreszcie, po raz trzeci wykonujemy analogiczne czynności, aby wygenerować pliki testowe odpowiadające znakom o 70% poziomie zaszumienia:

```
font_generator.py 128 128 28 10 times.ttf a 70 test_patterns_70
font_generator.py 128 128 28 10 times.ttf b 70 test_patterns_70
font_generator.py 128 128 28 10 times.ttf c 70 test_patterns_70
```

Po trzykrotnym wykonaniu programu `font_generator.py` z podanymi wyżej parametrami linii poleceń, w podkatalogu `test_patterns_70` powinny znaleźć się trzy pliki wyjściowe o rozdzielczościach 128x128 pikseli każdy, reprezentujące obrazy liter testowych o 70% poziomie zaszumienia losowego, które mogłyby wyglądać np. w następujący sposób:

a.png b.png c.png



Ponadto, w tym samym podkatalogu `test_patterns_70`, w wyniku wszystkich trzech wywołań programu `font_generator.py` wygenerowany zostanie pojedynczy tekstowy plik konfiguracyjny `description.txt`, który, będzie miał następującą postać:

Zawartość pliku tekstowego `description.txt` w podkatalogu `test_patterns_70`:

```
a.png:letter a, noise level 70%
b.png:letter b, noise level 70%
c.png:letter c, noise level 70%
```

W ten oto sposób przygotowane zostały wszystkie dane potrzebne do uruchomienia i zbadania skuteczności procesu rozpoznawania znaków przy użyciu sieci MADALINE.

Kolejnym krokiem jest już trzykrotne uruchomienie programu `madaline_ocr.py` w celu zbadania skuteczności i poprawności procesu rozpoznawania znaków przy użyciu sieci MADALINE dla znaków zaszumionych losowo w stopniach 30% i 70%, a także w stopniu 0% (tzn. dla samych znaków wzorcowych), co służy przetestowaniu poprawności działania programu `madaline_ocr.py`. Korzystając z przygotowanych wcześniej danych (tzn. plików obrazów i konfiguracyjnych plików tekstowych) uruchamiamy najpierw program `madaline_ocr.py` następującymi parametrami wywołania:

```
madaline_ocr.py training_patterns test_patterns_30
```

Program odczytuje wszystkie niezbędne dane umieszczone w podkatalogach `training_patterns` oraz `test_patterns_30` i generuje komunikat, wyświetlając go w oknie konsoli. Komunikat ten mógłby wyglądać w następujący sposób:

```
letter a, noise level 30% -> letter a, noise level 0%, confidence: 0.610
letter b, noise level 30% -> letter b, noise level 0%, confidence: 0.665
letter c, noise level 30% -> letter c, noise level 0%, confidence: 0.582
```

Komunikat ten zawiera pełen zestaw informacji stanowiących podsumowanie skuteczności procesu rozpoznawania znaków przy użyciu sieci MADALINE dla znaków zaszumionych losowo w stopniu 30%. Komunikat ten może stać się częścią raportu przygotowanego w ramach wykonania zadania projektowego.

Następnie wykonujemy analogiczne, co wyżej, czynności uruchamiając program `madaline_ocr.py` z następującymi parametrami wywołania:

```
madaline_ocr.py training_patterns test_patterns_70
```

Program odczytuje wszystkie niezbędne dane umieszczone w podkatalogach `training_patterns` oraz `test_patterns_70` i generuje komunikat, wyświetlając go w oknie konsoli. Komunikat ten mógłby wyglądać w następujący sposób:

```
letter a, noise level 70% -> letter a, noise level 0%, confidence: 0.382
letter b, noise level 70% -> letter b, noise level 0%, confidence: 0.420
letter c, noise level 70% -> letter c, noise level 0%, confidence: 0.342
```

Komunikat ten zawiera pełen zestaw informacji stanowiących podsumowanie skuteczności procesu rozpoznawania znaków przy użyciu sieci MADALINE dla znaków zaszumionych losowo w stopniu 70%. Podobnie jak wcześniej, komunikat ten może stać się częścią raportu przygotowanego w ramach wykonania zadania projektowego.

Ostatnie, trzecie, uruchomienie programu `madaline_ocr.py` służy przetestowaniu nie tyle skuteczności, co poprawności działania przygotowanych programów projektowych. W tym celu testujemy zaimplementowaną sieć MADALINE dla kształtów liter wzorcowych, uruchamiając program `madaline_ocr.py` z następującymi parametrami wywołania:

```
madaline_ocr.py training_patterns training_patterns
```

Program odczytuje wszystkie niezbędne dane umieszczone w podkatalogu `training_patterns`, traktując jego zawartość zarówno jako dane służące do utworzenia jak i przetestowania działania stworzonej sieci MADALINE. W tym wypadku powinniśmy otrzymać już konkretny komunikat o następującej postaci:

```
letter a, noise level 0% -> letter a, noise level 0%, confidence: 1.000
letter b, noise level 0% -> letter b, noise level 0%, confidence: 1.000
letter c, noise level 0% -> letter c, noise level 0%, confidence: 1.000
```

potwierdzający poprawności działania przygotowanych programów projektowych. Istotnie tak jest, ponieważ w przypadku gdy testujemy sieć MADALINE dla znaków testowych mających w istocie dokładnie takie same kształty jak znaki użyte do konstrukcji sieci, każdy ze znaków testowych powinien być rozpoznany prawidłowo ze współczynnikiem pewności równym dokładnie wartości 1 (ze względu na normalizację wektorów wejściowych i wagowych sieci – patrz opracowanie teoretyczne dotyczące sposobu działania sieci MADALINE podane w materiałach do zjazdu nr 1).

Oczywiście, podobnie jak wcześniej, komunikat wygenerowany dla tego przypadku powinien stać się częścią raportu przygotowanego w ramach wykonania zadania projektowego.

Na tym zakończymy omawianie przykładowego scenariusza użycia stworzonych aplikacji projektowych, w następnym podpunkcie podana zostanie specyfikacja zadania projektowego.

1.5.2 Specyfikacja zadania projektowego

Na podstawie materiałów podanych w Rozdziale 1 należy:

1. Stworzyć dwa skrypty w języku Python, implementujące i testujące poprawność działania prostego systemu rozpoznawania znaków opartego o wykorzystanie sieci neuronowej typu MADALINE. Wymienione wyżej skrypty to:

   - programu `font_generator.py`, służący do generowania plików graficznych zawierających próbki czcionek przeznaczonych do późniejszego rozpoznawania, reprezentujących pojedyncze litery o rożnym stopniu zaszumienia obrazu,

   - programu `madaline_ocr.py`, służący do rozpoznawania pojedynczych próbek czcionek na podstawie obrazów wygenerowanych za pomocą skryptu `font_generator.py`.

2. Przetestować skuteczność procesu rozpoznawania znaków dla znaków o stopniach zaszumienia, odpowiednio `10%`, `30%`, `50%`, i `90%`.

3. Przetestować poprawność procesu rozpoznawania znaków dla szczególnego przypadku zestawu znaków testowych o stopniu zaszumienia losowego równym `0%`.

4. Na podstawie wyników uzyskanych w przeprowadzonych testach, sporządzić raport z zadania projektowego zgodnie z szablonem sprawozdania podanym w materiałach do kursu.

5. Dla osób chętnych można przeprowadzić dodatkowe testy dla różnych rodzajów czcionek treningowych i testowych (o rożnym stopniu zaszumienia losowego), pokazujące efektywność stworzonego rozwiązania w przypadku gdy kształty czcionek treningowych nie odpowiadają dokładnie tym używanym podczas testowania systemu rozpoznawania (co jest sytuacją dość typową w przypadku realnych systemów typu *Optical Character Recognition*).

2. Wielowarstwowy perceptron i metoda wstecznej propagacji błędu

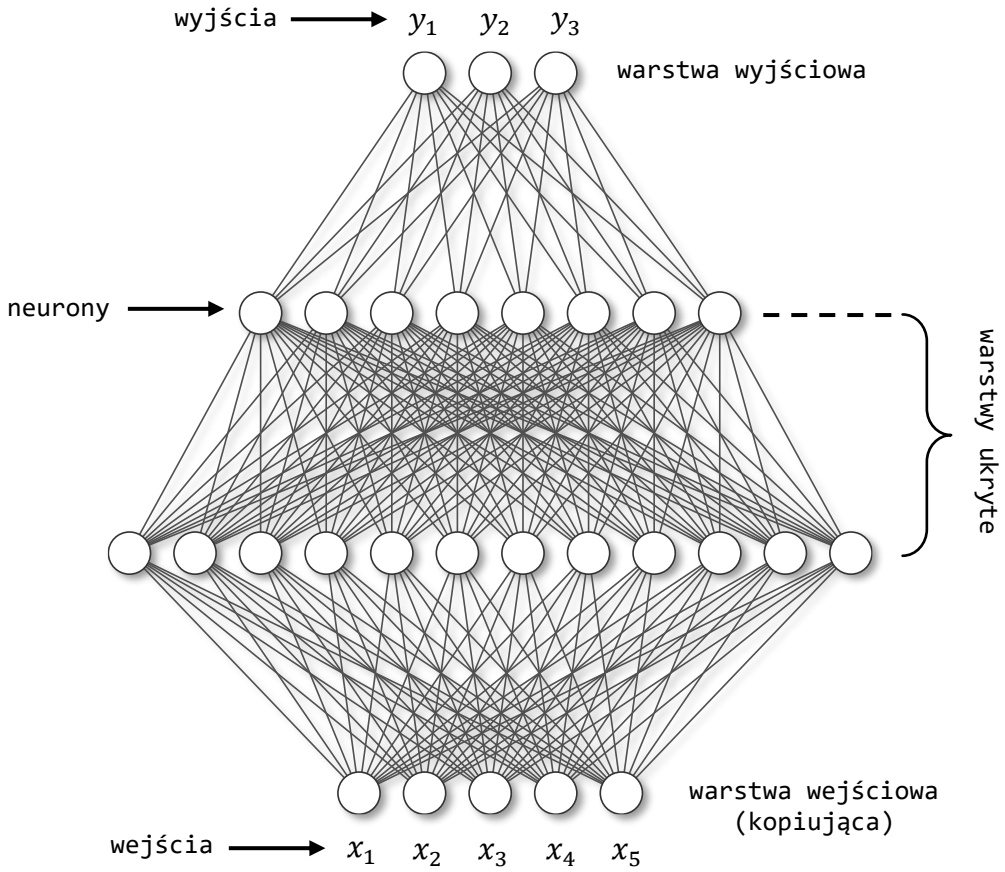
W niniejszym rozdziale zapoznamy się z siecią typu *wielowarstwowy perceptron* (ang. *Multilayer Perceptron*) oraz najpopularniejszą metodą treningu tego typu sieci zwaną w literaturze *metodą wstecznej propagacji błędu* (ang. *Error Back Propagation Method*). W tym celu:

- przedstawiona zostanie architektura i omówiony będzie sposób działania sieci typu wielowarstwowy perceptron oraz zdefiniowany zostanie problem treningu tego typu sieci
- przedstawiona zostanie metoda wstecznej propagacji błędu dla perceptronu wielowarstwowego jako metoda treningowa dedykowana tej sieci
- przedstawiona zostanie przykładowa implementacja własna algorytmu wstecznej propagacji błędu dla dwuwarstwowej sieci autoasocjacyjnej z warstwami liniowymi i nieliniowymi oraz neuronem obciążającym, czyli tak zwanym *bias*-em,
- przedstawiona zostanie implementacja wielowarstwowego perceptronu w zagadnieniu klasyfikacji obiektów z użyciem bibliotek programistycznych `Keras` i `scikit-learn`,
- omówiona zostanie specyfikacja propozycji zadań projektowych.

Wymienione wyżej zagadnienia i przykłady zastosowania sieci typu wielowarstwowy perceptron, zaprezentowane w niniejszym rozdziale, będą stanowić dobrą bazę do realizacji bardziej zaawansowanych zadań w dziedzinie aproksymacji funkcji złożonych, rozpoznawania i klasyfikacji obiektów, predykcji szeregów czasowych czy realizacji zadań sterownia różnego rodzaju urządzeniami technicznymi i procesami technologicznymi w oparciu o wykorzystanie sieci o architekturze wielowarstwowego perceptronu.

2.1 Architektura i sposób działania sieci typu wielowarstwowy perceptron

Sieć typu wielowarstwowy perceptron (ang. *Multilayer Perceptron*), w skrócie sieć MLP, jest jednokierunkową siecią neuronową składającą się z wielu warstw neuronów posiadających dowolnie wybrane funkcje aktywacji w ramach pojedynczej warstwy, o strukturze połączeń pokazanej na Rys 2.1.



Rysunek 2.1. *Architektura sieci neuronowej typu wielowarstwowy perceptron.*

Sieć taka składa się z kopiującej *warstwy wejściowej*, która nie podlega treningowi, zaś jej jedynym zadaniem jest propagacja składowych wektora wejściowego $\boldsymbol{x} = [\ x_1, x_2, \dots, x_N\ ]$ do wejść neuronów pierwszej warstwy sieci. Kolejne warstwy nazywamy *warstwami ukrytymi*, których liczba może być w ogólności dowolna, jednak najczęściej spotyka się sieci o jednej warstwie ukrytej, gdyż już tego typu sieci, na mocy odpowiednich twierdzeń matematycznych, są w stanie aproksymować dowolne funkcje analityczne z dowolnie dużą dokładnością, zależną od liczby neuronów obecnych w warstwie ukrytej. Ostatnią warstwą sieci MLP jest *warstwa wyjściowa*. Wyjścia neuronów tej warstwy stanowią współrzędne wektora wyjściowego $\boldsymbol{y} = [\ y_1, y_2, \dots, y_N\ ]$ całej sieci. Sieć MLP jest siecią jednokierunkową, co oznacza, że przepływ sygnałów w ramach sieci odbywa się tylko w jednym kierunku – od warstwy wejściowej, poprzez warstwy ukryte, aż do warstwy wyjściowej sieci. Funkcje aktywacji neuronów w ramach poszczególnych warstw sieci są z reguły takie same, natomiast mogą różnić się pomiędzy poszczególnymi warstwami, np. neurony warstwy ukrytej mogą mieć liniową funkcję akty-

wacji, a neurony warstwy wyjściowej funkcję sigmoidalną. Dobór funkcji aktywacji w poszczególnych warstwach sieci jest silnie zależny od konkretnego jej zastosowania.

Rozważmy teraz problem treningu sieci MLP. Załóżmy, że mamy do czynienia z siecią o N wejściach i M wyjściach, gdzie N oraz M są dowolnymi liczbami naturalnymi. Zdefiniujmy tzw. *zbiór treningowy* sieci następująco: $\Omega \stackrel{\text{def}}{=} \{ (\boldsymbol{x}^{(k)}, \boldsymbol{z}^{(k)}) \in \mathbb{R}^{N \times M};\ N, M \in \mathbb{N};\ k = 1, \ldots, K \}$. A zatem zbiór treningowy jest zbiorem K par wektorów $(\boldsymbol{x}^{(k)}, \boldsymbol{z}^{(k)})$, z których każda składa się z N – elementowego wektora wejściowego $\boldsymbol{x}^{(k)} = [\ x_1^{(k)}, x_2^{(k)}, \ldots, x_N^{(k)}\ ]$ oraz M – elementowego wektora wartości pożądanych na wyjściach sieci $\boldsymbol{z}^{(k)} = [\ z_1^{(k)}, z_2^{(k)}, \ldots, z_M^{(k)}\ ]$. Dla ustalonego $k \in \{ 1, \ldots, K \}$ para wektorów $(\boldsymbol{x}^{(k)}, \boldsymbol{z}^{(k)})$, nazywana jest k – tym *wzorcem treningowym* sieci. Przedstawiając zbiór treningowy w schematyczny sposób możemy zapisać:

$$\Omega \stackrel{\text{def}}{=} \left\{ \begin{array}{ll} ([\ x_1^{(1)}, x_2^{(1)}, \ldots, x_N^{(1)}\ ], [\ z_1^{(1)}, z_2^{(1)}, \ldots, z_M^{(1)}\ ]), & \leftarrow \text{wzorzec treningowy 1} \\ ([\ x_1^{(2)}, x_2^{(2)}, \ldots, x_N^{(2)}\ ], [\ z_1^{(2)}, z_2^{(2)}, \ldots, z_M^{(2)}\ ]), & \leftarrow \text{wzorzec treningowy 2} \\ ([\ x_1^{(3)}, x_2^{(3)}, \ldots, x_N^{(3)}\ ], [\ z_1^{(3)}, z_2^{(3)}, \ldots, z_M^{(3)}\ ]), & \leftarrow \text{wzorzec treningowy 3} \\ \vdots \quad\quad\quad\quad\quad\quad \vdots & \vdots \quad\quad\quad\quad\quad\quad\ \ \vdots \\ ([\ x_1^{(K)}, x_2^{(K)}, \ldots, x_N^{(K)}\ ], [\ z_1^{(K)}, z_2^{(K)}, \ldots, z_M^{(K)}\ ]) & \leftarrow \text{wzorzec treningowy K} \end{array} \right\}$$

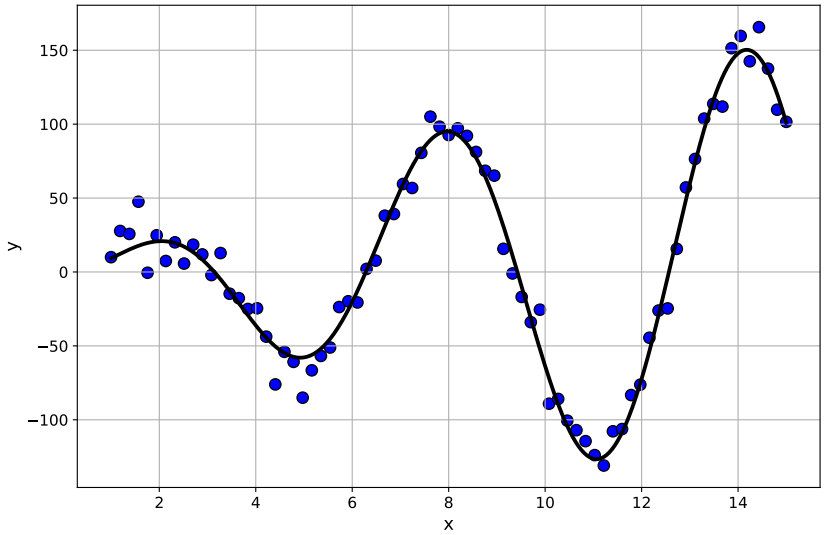
wektory wejściowe — wektory oczekiwane

zbiór treningowy

Dla tak zdefiniowanego zbioru uczącego zadanie treningu sieci neuronowej MLP można zapisać formalnie w następujący sposób:

$$\underset{\boldsymbol{w} \in \mathbb{R}^L}{\operatorname{argmin}} \{\ E(\boldsymbol{w}) \stackrel{\text{def}}{=} \frac{1}{2} \sum_{k=1}^{K} \sum_{i=1}^{M} (z_i^{(k)} - y_i^{(k)})^2\ \}, \tag{2.1}$$

gdzie wektor $\boldsymbol{w} \in \mathbb{R}^L$ jest (dowolnie uporządkowanym) wektorem wszystkich wag rozważanej sieci, $L \in \mathbb{N}$ jest liczbą wszystkich wag tejże sieci, zaś zdefiniowana we wzorze (2.1) funkcja $E : \mathbb{R}^L \to \mathbb{R}$ jest nazywana *funkcją błędu*. A zatem można powiedzieć, że celem treningu wielowarstwowego perceptronu jest znalezienie (jakąkolwiek metodą) dowolnego zestawu wag $\boldsymbol{w}$ rozważanej sieci, dla którego suma kwadratów różnic pomiędzy wyjściami sieci i odpowiadającymi im wartościami oczekiwanymi wzorców treningowych, po wszystkich wzorcach uczących i wszystkich wyjściach rozważanej sieci, osiąga wartość minimalną. Znalezienie takiego zestawu wag oznacza, że po zakończeniu procesu treningu wyjścia y_i sieci będą najbliższe, tak jak to tylko możliwe, odpowiadającym im wartościom oczekiwanym z_i, zadanym we wzorcach treningowych, w odpowiedzi na sekwencje wejściowe x_i zawarte w tych wzorcach. Zadanie to nosi nazwę matematyczną nazwę *problemu aproksymacji funkcji*. A zatem sieci typu MLP nazywane są często uniwersalnymi aproksymatorami adaptacyjnymi. Schematyczne przedstawienie problemu aproksymacji funkcji pokazane jest na Rys. 2.2. W kontekście sieci MLP pojedyncze, niebieskie punkty na Rys. 2.2 symbolizują wzorce treningowe (których współrzędne x-owe można utożsamiać z wektorami wejściowymi wzorców treningowych, zaś współrzędne y-owe z wektorami oczekiwanymi tych wzorców), zaś ciągła linia przybliżająca kształt zbioru punktów wzorcowych symbolizuje odpowiedź wytrenowanej sieci.



Rysunek 2.2. *Schematyczne przedstawienie problemu aproksymacji funkcji.*

Warto na koniec niniejszego punktu wspomnieć o tym, że wiele zadań rozwiązywanych bardzo często metodami sztucznej inteligencji, takich jak rozpoznawanie i klasyfikacja obiektów, grupowanie wzorców, predykcja szeregów czasowych czy realizacji zadań sterownia różnego rodzaju urządzeniami technicznymi i procesami technologicznymi, daje się sprowadzić do zadania aproksymacji. Stąd też sieci MLP są uznawane za wyjątkowo uniwersalne i stosowane zdecydowanie najczęściej, spośród innych znanych typów sieci neuronowych, w zadaniach ogólnie pojętej inteligentnej analizy danych.

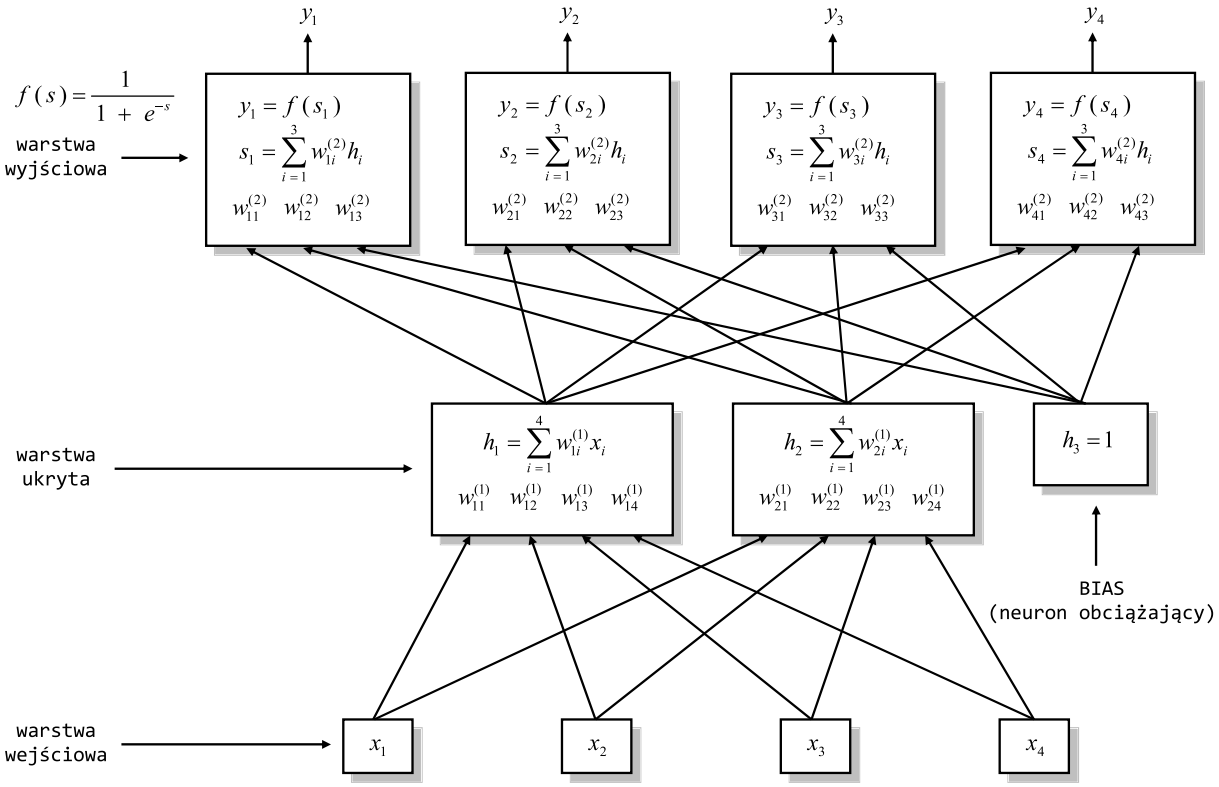
W tym miejscu zakończmy rozważania dotyczące architektury i sposób działania sieci typu wielowarstwowy perceptron. Kolejny podrozdział poświęcony będzie najpopularniejszej ze znanych metodzie treningu sieci MLP, tzw. *metodzie wstecznej propagacji błędu*.

2.2 Metoda wstecznej propagacji błędu treningu sieci MLP

Metoda wstecznej propagacji błędu, wraz z jej wieloma odmianami i wariantami, jest niewątpliwie jedną z najbardziej popularnych metod treningu wielowarstwowych sieci neuronowych. Podobnie jak, omawiana podczas Zjazdu nr 1, Reguła Delta należy do gradientowych technik optymalizacyjnych i opiera się dokładnie na tym samym pomyśle, co wspomniana reguła. Metoda ta zawdzięcza swoją popularność prostocie i potencjalnej efektywności implementacyjnej, gdyż pozwala na wykorzystanie istniejącej struktury sieci do przeprowadzenia procesu optymalizacji, co powoduje również, że nadaje się potencjalnie do wysoko efektywnych obliczeniowo i czasowo, równoległych realizacji zarówno programowych jak i sprzętowych. W niniejszym rozdziale przedstawimy tę metodę na prostym przykładzie sieci dwuwarstwowej z pojedynczym wzorcem treningowym, aby później rozszerzyć rozważania na przypadek ogólny.

Aby rozpocząć analizę działania metody wstecznej propagacji błędu, załóżmy, że mamy do czynienia z siecią dwuwarstwową MLP o architekturze uwidocznionej na Rys. 2.3. Sieć ta składa się z dwóch warstw adaptacyjnych (nie licząc kopiującej warstwy wejściowej, która nie podlega treningowi), tzn. warstwy ukrytej i wyjściowej. Neurony warstwy ukrytej posiadają identycznościową funkcję aktywacji, to znaczy, że ich wyjścia są bezpośrednimi sumami ważonymi wejść tych neuronów i ich wag. Neurony warstwy wyjściowej zaś posiadają sigmoidalną

funkcję aktywacji. Dodatkowo, ostatnie ważone wejście każdego z neuronów warstwy wyjściowej połączone jest z wyjściem *neuronu obciążającego*, tzw. *bias*-u. Neuron ten jest prostym neuronem kopiującym, który przez cały czas działania sieci podaje wartość 1 na wejścia neuronów z nim połączonych. Rola tego neuronu zostanie omówiona w dalszej części kursu. Wagi neuronów połączonych z bias-em również podlegają adaptacji w procesie uczenia sieci.



Rysunek 2.3. *Architektura przykładowej dwuwarstwowej sieci neuronowej typu MLP.*

Załóżmy ponadto, że zbiór uczący sieci posiada tylko jeden wzorzec treningowy zawierający wyłącznie wektory binarne (tzn. takie których elementami są wyłącznie zera lub jedynki). A zatem jego postać będzie następująca: $\Omega \stackrel{\text{def}}{=} \{ ([x_1, x_2, x_3, x_4], [z_1, z_2, z_3, z_4]) \in \{ 0, 1 \}^2 \}$. Zgodnie z tym, co powiedziane zostało w poprzednim podpunkcie, celem treningu sieci jest znalezienie dowolnego zestawu wag $\boldsymbol{w}$ rozważanej sieci, dla którego funkcja błędu będzie osiągała wartość minimalną, czyli w tym przypadku wartość 0. To znaczy, że po zakończeniu treningu sieci na jej wyjściach powinny pojawić się wartości maksymalnie zbliżone do wartości obecnych w wektorze wartości oczekiwanych $\boldsymbol{z} = [z_1, z_2, z_3, z_4]$, jeśli na jej wejścia podane zostaną wartości pochodzące z wektora wejściowego $\boldsymbol{x} = [x_1, x_2, x_3, x_4]$ wzorca treningowego. W przypadku naszej sieci przykładowej, funkcja błędu będzie miała następującą, uproszczoną ze względu na obecność pojedynczego wzorca treningowego w zbiorze uczącym, postać:

$$E (\boldsymbol{w}) = \sum_{i=1}^{4} (z_i - y_i)^2, \tag{2.2}$$

gdzie dla $i = 1, \ldots 4$, z_i oznaczają składowe wektora wartości oczekiwanych wzorca treningowego, zaś y_i są wartościami na wyjściach sieci uzyskanymi po przejściu sygnału wzorcowego $[x_1, x_2, x_3, x_4]$ dla ustalonego wektora $\boldsymbol{w} \in \mathbb{R}^{20}$ wszystkich wag sieci.

Stosując dokładnie ten sam pomysł, co w przypadku Reguły Delta dla pojedynczego neuronu, który poznaliśmy w trakcie Zjazdu nr 1, możemy posłużyć się metodą gradientową minimalizacji funkcji błędu E w przestrzeni wag sieci, aby iteracyjnie odnaleźć interesujący nas zestaw jej wag, dla którego funkcja E osiąga wartość minimalną, co, jak wspomnieliśmy wcześniej, jest właśnie celem treningu sieci. Jedynym zagadnieniem jakie musimy rozwiązać, aby zastosować tę metodę, jest odnalezienie postaci ujemnego gradientu funkcji błędu E dla dowolnego punktu $\boldsymbol{w}$ przestrzeni wag sieci. Dysponując postacią tego gradientu można już w łatwy sposób dokonać korekty wag sieci tak, aby zmniejszyć wartość funkcji błędu w kolejnym kroku iteracyjnym procesu treningu sieci – wystarczy (zgodnie z tym, co mówiliśmy w trakcie Zjazdu nr 1) do aktualnego wektora wag sieci $\boldsymbol{w}$ dodać, przeskalowaną przez małą wartość dodatnią η (krok uczący), wartość ujemnego gradientu funkcji błędu E w punkcie $\boldsymbol{w}$. Spróbujmy zatem znaleźć postać rzeczonego gradientu. Wprowadźmy w tym celu odpowiednie oznaczenia związane ze strukturą naszej sieci i samą metodą treningu:

- x_i – wyjście i-tego neuronu znajdującego się w warstwie wejściowej sieci,
- h_i – wyjście i-tego neuronu znajdującego się w warstwie ukrytej sieci,
- y_i – wyjście i-tego neuronu znajdującego się w warstwie wyjściowej sieci,
- z_i – pożądane wyjście i-tego neuronu znajdującego się w warstwie wyjściowej sieci,
- s_i – suma ważona i-tego neuronu znajdującego się w warstwie wyjściowej sieci,
- $w_{ij}^{(k)}$ – j-ta waga i-tego neuronu k-tej warstwy sieci,
- $f(\cdot)$ oraz $f'(\cdot)$ – funkcja sigmoidalna oraz jej pochodna,
- $\delta_i^{(k)}$ – błąd odpowiedzi i-tego neuronu znajdującego się w k-tej warstwie sieci,
- η – stały krok treningowy będący małą liczbą dodatnią.

Przypomnijmy, że gradient funkcji błędu w przestrzeni wag sieci jest wektorem pochodnych cząstkowych tej funkcji po wszystkich współrzędnych wagowych sieci, a zatem jawną postać ujemnego gradientu funkcji błędu dla naszej sieci przykładowej możemy zapisać tak:

$$-\nabla E(\boldsymbol{w}) = [\underbrace{-\frac{\partial E(\boldsymbol{w})}{\partial w_{11}^{(1)}}, -\frac{\partial E(\boldsymbol{w})}{\partial w_{12}^{(1)}}, \ldots, -\frac{\partial E(\boldsymbol{w})}{\partial w_{24}^{(1)}}}_{\text{wagi warstwy ukrytej}}, \underbrace{-\frac{\partial E(\boldsymbol{w})}{\partial w_{11}^{(2)}}, -\frac{\partial E(\boldsymbol{w})}{\partial w_{12}^{(2)}}, \ldots, -\frac{\partial E(\boldsymbol{w})}{\partial w_{43}^{(2)}}}_{\text{wagi warstwy wyjściowej}}] \quad (2.3)$$

Okazuje się, że dla wag warstwy wyjściowej naszej sieci przykładowej, współrzędne ujemnego gradientu funkcji błędu będą miały postać:

$$-\frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(2)}} = f'(s_i) \cdot (z_i - y_i) \cdot h_j, \; i = 1, \ldots, 4, \; j = 1, \ldots, 3. \quad (2.4)$$

Zdefiniujmy „błąd" popełniany na wyjściach warstwy wyjściowej sieci w następujący sposób:

$$\delta_i^{(2)} \stackrel{\text{def}}{=} f'(s_i) \cdot (z_i - y_i), \; i = 1, \ldots, 4. \quad (2.5)$$

Wtedy okazuje się, że dla wag warstwy ukrytej naszej sieci przykładowej, współrzędne ujem-

nego gradientu funkcji błędu będą miały postać:

$$-\frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(1)}} = \sum_{k=1}^{4} w_{ki}^{(2)} \delta_k^{(2)} \cdot x_j\,,\ i = 1,\dots,2,\ j = 1,\dots,4. \tag{2.6}$$

Podobnie, zdefiniujmy „błąd" popełniany na wyjściach warstwy ukrytej sieci następująco:

$$\delta_i^{(1)} \stackrel{\text{def}}{=} \sum_{k=1}^{4} w_{ki}^{(2)} \delta_k^{(2)}\ ,\ i = 1,\dots,2. \tag{2.7}$$

Wtedy postaci współrzędnych ujemnego gradientu funkcji błędu dla warstwy wyjściowej sieci możemy zapisać w następujący sposób:

$$-\frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(2)}} = \delta_i^{(2)} \cdot h_j,\ i = 1,\dots,4,\ j = 1,\dots,3, \tag{2.8}$$

zaś współrzędne ujemnego gradientu funkcji błędu dla warstwy ukrytej sieci będą miały następujące postaci:

$$-\frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(1)}} = \delta_i^{(1)} \cdot x_j,\ i = 1,\dots,2,\ j = 1,\dots,4. \tag{2.9}$$

Dysponując postaciami wszystkich współrzędnych ujemnego gradientu funkcji błędu można zapisać już regułę korekty każdej z wag sieci prowadzącą do zmniejszenia wartości funkcji błędu. I tak, dla warstwy ukrytej sieci dostajemy:

$$w_{ij}^{(1)} = w_{ij}^{(1)} - \eta \cdot \frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(1)}} = w_{ij}^{(1)} + \eta \cdot \delta_i^{(1)} \cdot x_j,\ i = 1,\dots,2,\ j = 1,\dots,4, \tag{2.10}$$

zaś dla warstwy wyjściowej sieci reguła ta przyjmie postać:

$$w_{ij}^{(2)} = w_{ij}^{(2)} - \eta \cdot \frac{\partial E(\boldsymbol{w})}{\partial w_{ij}^{(2)}} = w_{ij}^{(2)} + \eta \cdot \delta_i^{(2)} \cdot h_j,\ i = 1,\dots,4,\ j = 1,\dots,3. \tag{2.11}$$

W ten oto sposób wyprowadziliśmy *de facto* metodę wstecznej propagacji błędu dla naszej przykładowej sieci z Rys. 2.3 i pojedynczego wzorca uczącego.

Warto teraz przekuć powyższe wyprowadzenie na algorytm treningowy naszej przykładowej sieci neuronowej. Pseudokod tego algorytmu znajduje się na schemacie 2.1.

1) Ustaw wszystkie wagi sieci na wartości losowe z założonego z góry przedziału $[\ \omega_1\ ,\omega_2\ ]$ zawartego w zbiorze liczb rzeczywistych.

2) Dla epoki treningowej $l = 1, \dots, L$ powtarzaj wykonywanie kroków od 3) do 10).

3) Ustaw wektor wejściowy $\boldsymbol{x} = [\ x_1,\ x_2,\ x_3,\ x_4\ ]$ wzorca treningowego na wejścia sieci.

4) Oblicz wyjścia neuronów warstwy ukrytej sieci wg wzoru:

$$h_i = \sum_{j=1}^{4} w_{ij}^{(1)} x_j = w_{i1}^{(1)} x_1 + w_{i2}^{(1)} x_2 + w_{i3}^{(1)} x_3 + w_{i4}^{(1)} x_4\ ,\ \ i = 1, 2. \tag{2.1.1}$$

5) Oblicz sumy ważone neuronów warstwy wyjściowej sieci wg wzoru:

$$s_i = \sum_{j=1}^{3} w_{ij}^{(2)} h_j = w_{i1}^{(2)} h_1 + w_{i2}^{(2)} h_2 + w_{i2}^{(2)} h_3,\ \ i = 1, \dots, 4, \tag{2.1.2}$$

pamiętając, że wyjście $h_3 = 1$ jest wyjściem neuronu obciążającego.

6) Oblicz wyjścia neuronów warstwy wyjściowej sieci wg wzoru:

$$y_i = f(s_i),\ \ i = 1, \dots, 4, \tag{2.1.3}$$

gdzie $f(s) = \frac{1}{1 + e^{-s}}$ jest sigmoidalną funkcją aktywacji neuronów warstwy wyjściowej.

7) Oblicz błędy warstwy wyjściowej sieci wg wzoru:

$$\delta_i^{(2)} = f'(s_i) \cdot (\ z_i - y_i\ )\ ,\ \ i = 1, \dots, 4, \tag{2.1.4}$$

gdzie $f'(s) = f(s) \cdot (1 - f(s))$ jest pochodną funkcji sigmoidalnej, y_i są wyjściami sieci obliczonymi w punkcie 6), zaś z_i są współrzędnymi wektora $\boldsymbol{z} = [\ z_1,\ z_2,\ z_3,\ z_4\ ]$ wartości pożądanych na wyjściach sieci wzorca treningowego.

8) Korzystając z obliczonych w punkcie 7) wartości błędów $\delta_i^{(2)}$ warstwy wyjściowej sieci, oblicz błędy warstwy ukrytej sieci wg wzoru (propagacja wsteczna błędów):

$$\delta_i^{(1)} = \sum_{k=1}^{4} w_{ki}^{(2)} \delta_k^{(2)}\ ,\ \ i = 1, \dots, 2. \tag{2.1.5}$$

9) Korzystając z obliczonych w punktach 7) oraz 8) wartości błędów $\delta_i^{(2)}$ warstwy wyjściowej i wartości błędów $\delta_i^{(1)}$ warstwy ukrytej sieci, dokonaj korekty wag sieci wg wzorów:

$$w_{ij}^{(1)} = w_{ij}^{(1)} + \eta \cdot \delta_i^{(1)} \cdot x_j,\ \ i = 1, \dots, 2,\ \ j = 1, \dots, 4,\ \text{dla warstwy ukrytej sieci,} \tag{2.1.6a}$$

$$w_{ij}^{(2)} = w_{ij}^{(2)} + \eta \cdot \delta_i^{(2)} \cdot h_j,\ \ i = 1, \dots, 4,\ \ j = 1, \dots, 3,\ \text{dla warstwy wyjściowej sieci,} \tag{2.1.6b}$$

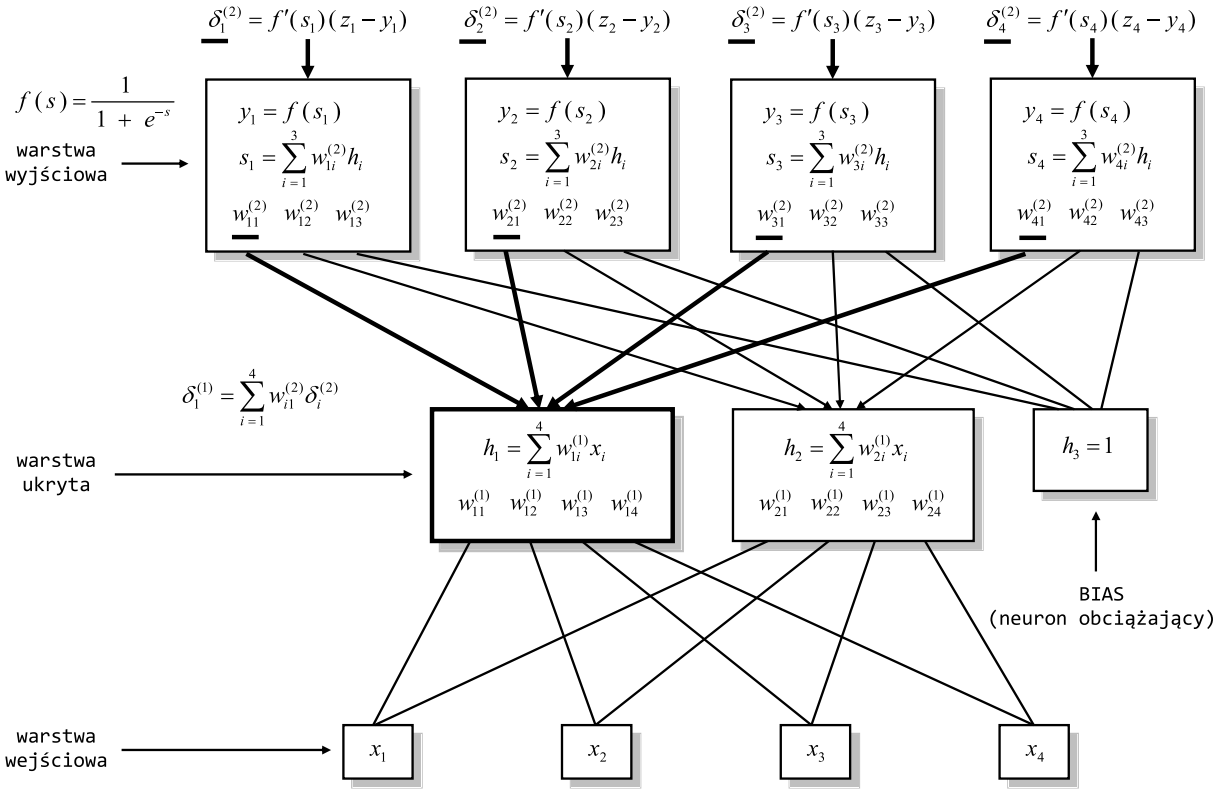
gdzie η jest stałym krokiem treningowym będącym małą rzeczywistą liczbą dodatnią, x_j są współrzędnymi wektora wejściowego sieci, h_j są wyjściami warstwy ukrytej sieci obliczonymi w pkt. 4), pamiętając o tym, że wyjście $h_3 = 1$ jest wyjściem neuronu obciążającego.

10) Powróć do kroku 2) rozpoczynając kolejną epokę treningu sieci.

Algorytm 2.1. *Algorytm wstecznej propagacji błędu dla dwuwarstwowej sieci MLP z Rys. 2.3 i pojedynczego wzorca treningowego.*

Alg. 2.1 jest implementacją metody wstecznej propagacji błędu dla dwuwarstwowej sieci MLP z rysunku 2.3 i pojedynczego wzorca treningowego. Aby dokładnie zrozumieć przepływ sygnałów w sieci podczas realizacji tego algorytmu oraz dokładnie uświadomić sobie skąd wzięła się jego nazwa prześledźmy w formie schematycznej i skomentujmy najważniejsze kroki tego algorytmu.

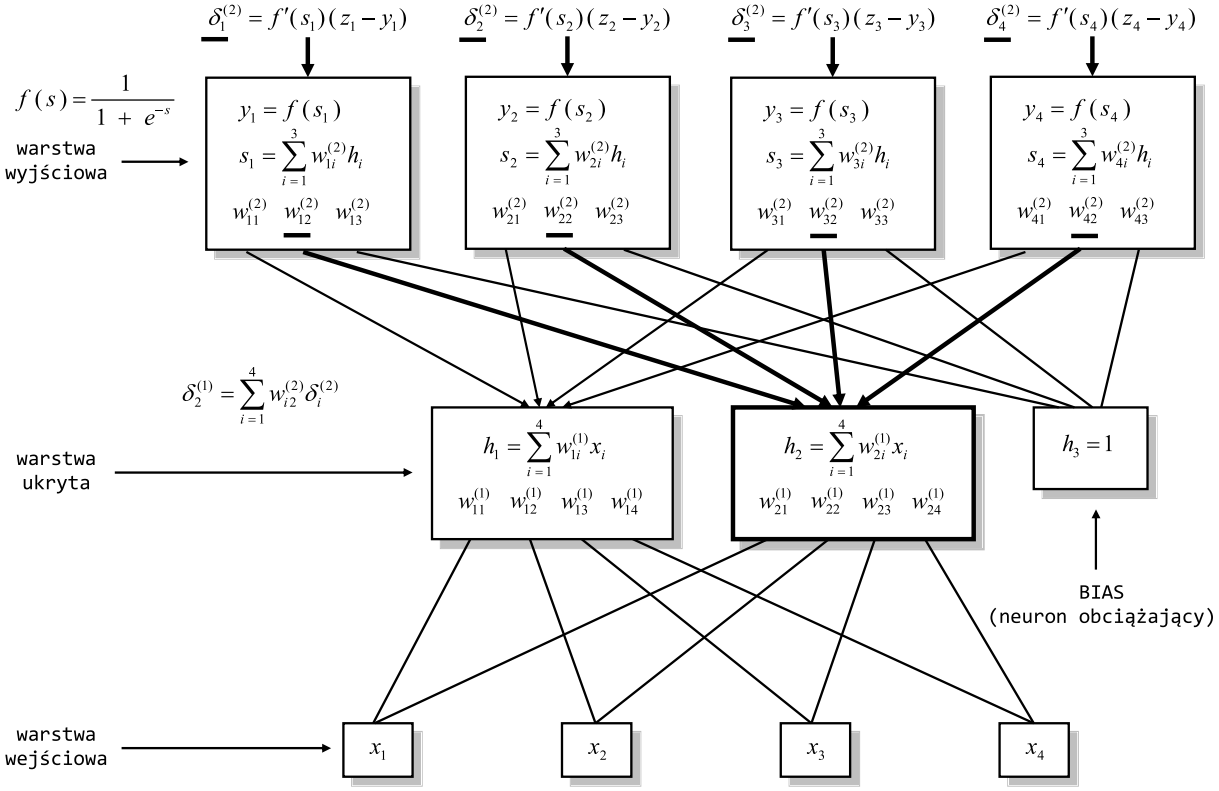
Najpierw, w punkcie 1), ustawiamy wszystkie wagi sieci na wartości losowe z zadanego z góry przedziału. Następnie rozpoczynamy trening sieci, który polega na iteracyjnym (wielokrotnym) wykonywaniu kroków od 3) do 10), aż do osiągnięcia ostatniej, L–tej iteracji algorytmu. Pojedyncza iteracja procesu treningowego sieci nazywana jest *epoką* treningową. W krokach od 3) do 6) przesyłamy sygnał wejściowy $\boldsymbol{x} = [\, x_1,\, x_2,\, x_3,\, x_4\,]$ od wejść, poprzez neurony warstwy ukrytej, aż do wyjść sieci, obliczając ostatecznie wartości na wyjściach sieci y_i, $i = 1, \ldots, 4$. Proces ten dobrze obrazuje Rys. 2.3. Kroki 7) i 8) polegają na obliczeniu błędów na wyjściach neuronów warstwy wyjściowej, a następnie warstwy ukrytej sieci. To właśnie ten fragment algorytmu odpowiada za jego nazwę. Aby uzmysłowić sobie dlaczego, spójrzmy na Rys. 2.4 oraz 2.5 obrazujące strukturę przepływu sygnałów błędów podczas tego etapu działania algorytmu.



Rysunek 2.4. *Struktura przepływu sygnałów błędów podczas etapu propagacji tych sygnałów wstecz sieci (przypadek dla neuronów warstwy wyjściowej i pierwszego neuronu warstwy ukrytej).*

Jak widać z rysunku 2.4 błędy $\delta_i^{(2)}$, $i = 1, \ldots, 4$, dla warstwy wyjściowej sieci (podkreślone czarną kreską) mogą być wyznaczone bezpośrednio na podstawie znajomości wszystkich sygnałów obliczonych w punktach od 3) do 6) Alg. 2.1 podczas propagacji sygnałów wejściowych wprzód sieci. Błędy warstwy ukrytej $\delta_i^{(1)}$ natomiast (w przypadku Rys. 2.4 dotyczy to błędu $\delta_1^{(1)}$ pierwszego neuronu warstwy ukrytej) obliczane są poprzez niejako przesyłanie błędów warstwy wyjściowej $\delta_i^{(2)}$ wstecz sieci, po przemnożeniu ich przez odpowiednie wagi neuronów warstwy

wyjściowej, z którymi połączony jest neuron, dla którego wyznaczamy interesujący nas błąd (patrz pogrubione strzałki i podkreślone wagi dla pogrubionego prostokąta reprezentującego pierwszy neuron warstwy ukrytej dla którego wyznaczany jest błąd). Sytuacja będzie wyglądać całkowicie analogicznie dla neuronu drugiego warstwy ukrytej – jest ona zobrazowana na poniższym Rys. 2.5.



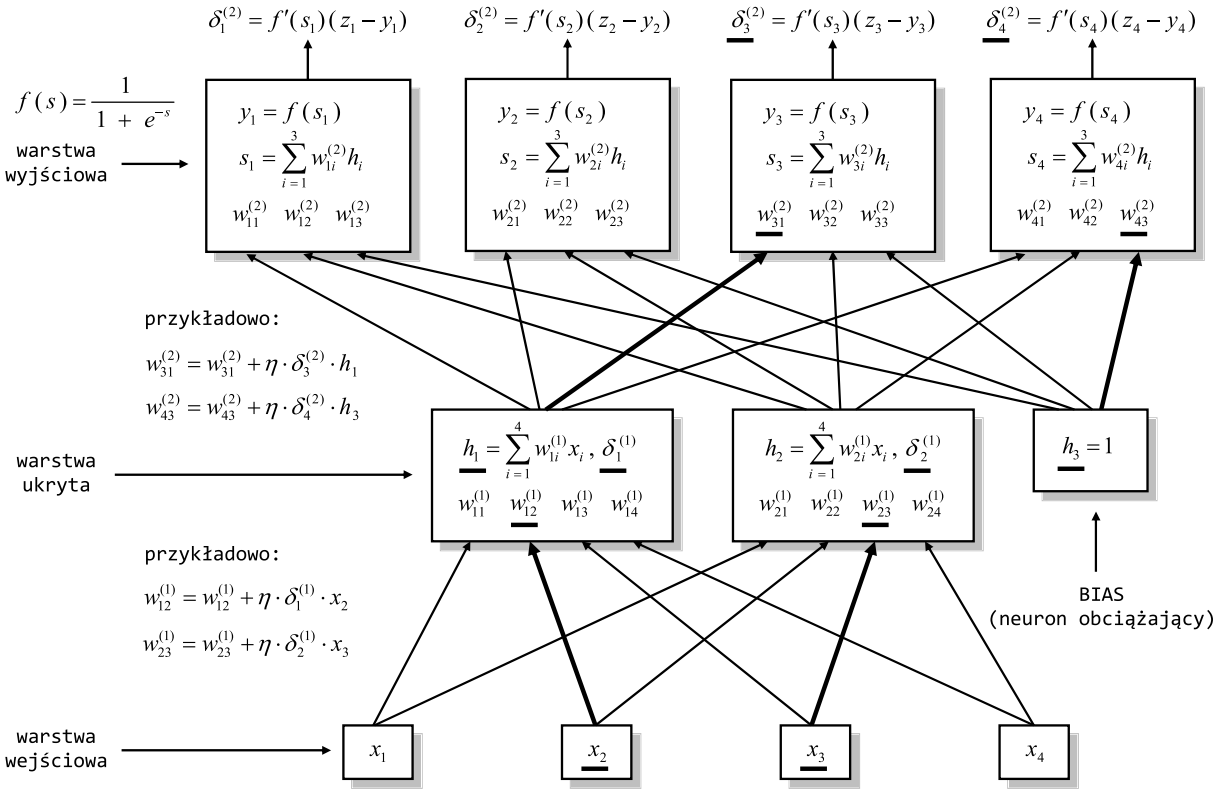
Rysunek 2.5. *Struktura przepływu sygnałów błędów podczas etapu propagacji tych sygnałów wstecz sieci (przypadek dla neuronów warstwy wyjściowej i drugiego neuronu warstwy ukrytej).*

Tutaj widać, że aby obliczyć błąd $\delta_2^{(1)}$ dla drugiego neuronu warstwy ukrytej należy przesyłać niejako błędy warstwy wyjściowej $\delta_i^{(2)}$ wstecz sieci, po przemnożeniu ich przez odpowiednie wagi neuronów warstwy wyjściowej połączone z neuronem drugim warstwy ukrytej.

To właśnie temu schematowi postępowania, pokazanemu obrazowo na Rys. 2.4 i 2.5, który ujawnia się podczas obliczania błędów warstw wewnętrznych sieci MLP, algorytm zawdzięcza swoją nazwę – metody wstecznej propagacji błędu.

Po obliczeniu wszystkich błędów dla warstwy wyjściowej i ukrytej sieci MLP, można przystąpić już do ostatniego etapu działania algorytmu w pojedynczej iteracji obliczeniowej, tzn. korekty wag sieci. Jest ona wykonywana w punkcie 9) algorytmu. Schemat postępowania podczas etapu korekty wag sieci algorytmu wstecznej propagacji błędu dla wybranych wag przykładowych z warstw ukrytej i wyjściowej sieci przedstawiony jest na Rys. 2.6. Widać tutaj, że podczas korekty wag wykorzystywane są już wszystkie sygnały, które zostały obliczone wcześniej podczas obydwu etapów propagacji sygnałów – wprzód oraz wstecz sieci. Korekty wag sieci można dokonać w dowolnej kolejności. Z Rys. 2.6 (a także oczywiście ze wzorów (2.1.6a) oraz (2.1.6b)), wynika, że dla dowolnej wagi (niezależnie od tego czy znajduje się ona w warstwie ukrytej czy wyjściowej sieci) jej nowa wartość będzie zmieniona o iloczyn współczynnika nauczania, błędu neuronu w którym korygowana waga się znajduje oraz wartość sygnału

wejściowego, który jest z tą wagą połączony. Korekta wag jest już ostatnim etapem działania pojedynczej iteracji obliczeniowej (epoki) algorytmu wstecznej propagacji błędu. Zgodnie z jego sformułowaniem, w tym momencie następuje skok do punktu 2) i proces rozpoczyna się od nowa, lecz tym razem wszystkie kroki będą powtórzone już dla skorygowanego zestawu wag sieci. Postępując w ten sposób odpowiednio dużą liczbę epok, funkcja błędu sieci MLP powinna osiągnąć zakładane minimum (najlepiej globalne), co oznacza, że proces treningowy osiągnął swój założony cel.



Rysunek 2.6. *Schemat postępowania podczas etapu korekty wag sieci algorytmu wstecznej propagacji błędu dla wybranych wag przykładowych z warstw ukrytej i wyjściowej sieci.*

W niniejszym podpunkcie wyprowadziliśmy, omówiliśmy i sformułowaliśmy algorytm metody wstecznej propagacji błędu dla prostego przykładu sieci dwuwarstwowej z pojedynczym wzorcem treningowym, aby poznać i zrozumieć zasadę jej działania. Metoda ta może zostać w analogiczny, bezpośredni sposób rozszerzona na przypadek sieci wielowarstwowych o dowolnych architekturach dla wielu wzorców treningowych. W następnym podpunkcie podamy jej pełną postać dla, najczęściej stosowanej w praktyce, sieci dwuwarstwowej uczonej na zbiorze wielu wzorców treningowych.